N° d'ordre : 014 / STI /TCO Année Universitaire : 2023 / 2024

UNIVERSITE D'ANTANANARIVO

ECOLE SUPERIEURE POLYTECHNIQUE

MENTION TELECOMMUNICATION

MEMOIRE

En vue de l'obtention

Du DIPLOME de LICENCE

Domaine : Sciences de l'Ingénieur

Mention : Télécommunication

Parcours : Système et Traitement d'Information (STI)

Par : RAMANANANDRO Alex Mathieu

CONCEPTION D'UNE INTERFACE HOMME-MACHINE CONTRÔLÉE PAR DES GESTES DE LA MAIN

Soutenu le **24 Octobre 2024** devant la Commission d'Examen composée de :

Président : Monsieur RAJAONARISON Tianandrasana Roméo

Examinateurs : Monsieur RANDRIAMANAMPY Samuel

Directeur de mémoire : Monsieur BOTO ANDRIANANDRASANA Jean Espérant

REMERCIEMENTS

Je tiens d'abord à rendre grâce à Dieu pour sa bienveillance infinie et pour m'avoir donné la force, la santé et la sagesse nécessaires à la réalisation de ce travail. Sans sa guidance, ce projet n'aurait pu aboutir.

Je souhaite exprimer ma profonde gratitude à Monsieur RAVELOMANANA Mamy Raoul, Professeur Titulaire et Président de l'Université d'Antananarivo. Son engagement en faveur de l'excellence académique a permis à de nombreux étudiants, dont moi-même, de poursuivre leurs études dans un environnement intellectuellement stimulant.

Mes sincères remerciements vont à Monsieur RAKOTOSAONA Rijalalaina, Professeur titulaire et Directeur de l'École Supérieure Polytechnique d'Antananarivo, pour m'avoir accueilli au sein de son établissement.

Je tiens également à remercier Monsieur RAJAONARISON Tianandrasana Roméo, Maître de Conférence-HDR en Télécommunication et Chef de la Mention Télécommunication, ainsi que Président du jury, pour son engagement dévoué au bon déroulement de ma formation au sein de cette mention et pour avoir présidé avec rigueur et bienveillance les travaux de soutenance.

Je remercie tout particulièrement Monsieur BOTO ANDRIANANDRASANA Jean Espérant, Directeur de ce mémoire, pour son encadrement attentif, ses orientations précieuses et son soutien technique tout au long de ce projet. Sa patience et son expertise ont grandement facilité l'aboutissement de ce travail.

Je remercie également Monsieur RANDRIAMANAMPY Samuel, membre du jury, pour avoir pris le temps d'examiner ce mémoire et pour ses remarques constructives, qui ont permis d'en améliorer la qualité.

Mes remerciements vont à l'ensemble des professeurs de la mention Télécommunication, pour leur dévouement et leur engagement envers la formation des étudiants. Grâce à leurs enseignements, j'ai pu acquérir des connaissances solides qui ont été indispensables à la réalisation de ce mémoire.

Enfin, je tiens à exprimer ma profonde gratitude à ma famille et à mes amis pour leur soutien indéfectible, leurs encouragements et leur présence tout au long de cette aventure. Leur appui moral a été une source inestimable de motivation.

TABLE DES MATIERES

REMERCIEMENTS i

TABLE DES MATIERES ii

LISTE DES NOTATIONS ET ABREVIATIONS vi

INTRODUCTION GENERALE 1

CHAPITRE 1 2

GENERALITES SUR LA RECONNAISSANCE DES GESTES 2

1.1 Introduction 2

1.2 Interaction Homme – Machine gestuelle 2

1.2.1 Communication gestuelle *2*

1.2.2 Types de Gestes *2*

1.2.3 Dispositifs d'interaction *3*

1.3 La Vision artificielle 5

1.3.1 Définition *5*

1.3.2 Mécanisme de la Vision Humaine *5*

1.3.3 Perception du couleur *5*

1.3.4 Principe de la vision artificielle *6*

1.4 Traitement d'image 6

1.4.1 Introduction *6*

1.4.2 Notion d'image *7*

1.4.3 Image numérique *7*

1.4.4 Caractéristiques d'une image numérique *8*

1.4.5 Les systèmes de traitement d'images *9*

1.4.6 Les techniques segmentation d'image pour la reconnaissance des gestes *10*

Conclusion 12

CHAPITRE 2 : 13

RÉSEAU NEURONAL POUR LA RECONNAISSANCE GESTUELLE 13

2.1 Introduction **13**

2.2 Historique **13**

2.3 Réseaux de neurone Artificiel **16**

2.3.1 Neurone biologique ***16***

2.3.2 Le Neurone Formel ***16***

2.3.3 Théorie du perceptron (OU, ET, OU exclusif) ***18***

2.3.4 Optimisation d'un Neurone Artificiel ***20***

2.3.4 Réseaux de neurone multicouche ***21***

2.4 Réseau de neurone convolutif **24**

2.4.1 Architecture d'une CNN ***24***

2.4.2 Couche convolution ***24***

2.4.3 Couche pooling ***26***

2.4.4 Architecture globale d'un réseau convolutionnel ***27***

2.5 Méthodologie et Processus de Développement **28**

2.5.1 Processus de Reconnaissance des Gestes ***28***

2.5.2 Préparation des Données ***29***

2.5.3 Pretraitement des Donnees : ***29***

2.5.4 Model de Construction ***30***

Conclusion **33**

CHAPITRE 3 ..... **34**

REALISATION DU SYSTEME HOMME-MACHINE CONROLE PAR DES GESTES **34**

3.1 Introduction **34**

3.2 Description de projet **34**

3.3 Architecture Globale du système : **35**

3.3.1 L'acquisition des images ***36***

3.3.2 La segmentation ***36***

3.3.3 Extraction des caractéristiques (troisième étapes) *37*

3.3.4 Classification et reconnaissance des gestes : *37*

3.4 Outils utilisés et environnement de développement **38**

3.4.1 Python *38*

3.4.2 OpenCv *38*

3.4.3 Tkinter *39*

3.4.5 Numpy *40*

3.4.6 MediaPipe *40*

3.4.7 Pyatougui *41*

3.4 Etapes et processus du projet **41**

3.4.1 Importer les packages nécessaires *42*

3.4.2 Initialiser les modèles *42*

3.4.3 Lire les images d'une webcam *42*

3.4.4 Détecter les points clés de la main *42*

3.4.5 Reconnaître les gestes de la main *43*

Conclusion **44**

CHAPITRE 4 : EVALUATION ET INTERPRETATION DES RESULTATS DU SYSTEME DE RECONNAISSANCE GESTUELLE **45**

4.1 Représentation de l'application **45**

4.1.1 Interface de l'application *45*

4.1.2 But de l'application *48*

4.2 Présentation des tests **49**

4.3 Métriques d'évaluation **51**

4.3.1 Analyse des Résultats *52*

4.3.2 FPS over Time ou Fréquence d'Images en fonction du temps *53*

4.3.3 Temps de Détection de la Main en Fonction du Temps *54*

4.4 Avantages et apports du système de contrôle par gestes de la main **56**

4.5 Limitations du Projet **56**

Conclusion **57**

CONCLUSION GENERALES **58**

ANNEXES **59**

ANNEXES 1 ..... **59**

NOTIONS ESSENTIELLES **59**

ANNEXE 2 ..... **59**

CODE SOURCE DE LA RECONNAISSANCE DE LA MAIN **59**

ANNEXE 2 ..... **61**

CODE SOURCE SUR LE SOURIS VIRTUEL **61**

ANNEXE 3 ..... **63**

BIBLIOGRAPHIE **65**

FICHE DE RENSEIGNEMENTS **67**

RESUME **68**

ABSTRACT **68**

LISTE DES NOTATIONS ET ABREVIATIONS

1. Minuscules latines

a poids synaptique de xi

c célérité de la lumière (3×10^8 m.s^{-1})

f fonction quelconque

xi Entrées de réseau de neurone

yi Sortie de réseau de neurone

2. Majuscules Latines

D Dimension

R Région d'une image

T Période de l'onde

3. Minuscules grecques

α Pas d'apprentissage

θ Valeur actuelle d'un poids dans un neurone formel

θ' Valeur corrigée d'un poids dans un neurone formel

ϕ

4. Abréviations

CCD Charge-Coupled Device (Capteur d'image

CNN Convolutional Neural Network

Conv1D Convolution 1D

FC Fully Connected (Couche entièrement connectée)

FPS frames per second

HID (Human Interface Device)

MLP Multi-Layer Perceptron

MSE Mean Squared Error (Erreur quadratique moyenne)

PMC Perceptron Multicouche

POOL Pooling(Couche de pooling dans un CNN)

ReLu Rectified Linear Unit

RGDS Registerable Gesture Dataset

RNN Recurrent Neural Network

RVB Rouge, Vert, Bleu

INTRODUCTION GENERALE

Dans un monde où la technologie numérique s'intègre de plus en plus dans notre quotidien, l'interaction homme-machine est devenue un enjeu majeur. L'évolution rapide de l'informatique a transformé nos modes de vie et soulevé de nouvelles questions sur la manière d'interagir avec les appareils. La technologie influence désormais la communication, le travail et nos activités quotidiennes, rendant les interactions plus intuitives. Pourtant, les interfaces traditionnelles, comme la souris, montrent leurs limites face à des contextes tels que la robotique et la réalité augmentée. C'est ainsi qu'est née l'étude sur la "**Conception d'une interface Homme-Machine contrôlée par des gestes de la main**", exploitant les progrès récents en vision artificielle et apprentissage profond.

Ces avancées apportent des solutions à plusieurs défis. La vision artificielle permet d'interpréter visuellement des informations en temps réel, et l'apprentissage profond améliore la reconnaissance des gestes, offrant des interactions naturelles et sans contact. En réponse à des problématiques comme celles posées par la pandémie de COVID-19, la reconnaissance gestuelle devient une alternative intuitive et hygiénique dans divers contextes, réduisant les points de contact physiques.

L'objectif principal de ce projet est de créer un système permettant de contrôler un ordinateur via des gestes, facilitant ainsi de nouvelles applications, allant de l'interaction avec des robots à la commande de jeux vidéo, ou encore à des systèmes d'assistance pour les personnes en situation de handicap. Ce système peut également être envisagé comme une solution à des problématiques sanitaires, en réduisant les points de contact physiques, ou encore comme un outil d'amélioration de l'accessibilité dans de multiples environnements.

Pour structurer ce travail, quatre points clés interconnectés seront développés. Nous commencerons par une première partie consacrée aux généralités sur la reconnaissance des gestes, où les concepts fondamentaux seront définis tout en explorant l'écosystème technologique associé. Ensuite, nous aborderons le cœur du sujet en présentant le réseau neuronal conçu pour la reconnaissance gestuelle, avec une analyse détaillée des méthodes et approches utilisées pour atteindre cet objectif. La troisième section portera sur la réalisation du système homme-machine contrôlé par des gestes, en décrivant l'architecture, les outils et les technologies employées. Enfin, nous conclurons par une analyse approfondie des résultats obtenus, accompagnée d'une interprétation visant à évaluer l'efficacité et les performances du modèle proposé.

CHAPITRE 1

GENERALITES SUR LA RECONNAISSANCE DES GESTES

1.1 Introduction

La reconnaissance des gestes de la main est un domaine clé dans l'interaction Homme-Machine, permettant de contrôler des systèmes informatiques de manière naturelle et intuitive. Imaginez pouvoir contrôler vos appareils d'un simple mouvement de la main, sans souris ni clavier. Dans ce chapitre, nous allons détailler les généralités des systèmes de reconnaissance des gestes.

1.2 Interaction Homme – Machine gestuelle

1.2.1 Communication gestuelle

Les gestes sont un moyen de communication essentiel pour l'être humain, permettant d'agir sur le monde et de transmettre des informations. Ils peuvent être réalisés avec différentes parties du corps, comme les mains, les bras ou la tête, parfois avec une intention de communiquer, parfois non. En plus de transmettre des messages, les gestes nous aident aussi à en recevoir.

Selon Cadoz [1], il existe trois fonctions principales des gestes de la main :

- La fonction ergotique : La main agit comme un moteur pour modifier le monde physique. Elle exerce des forces sur les objets pour les déplacer ou les transformer.
- La fonction épistémique : La main sert à percevoir des informations grâce au toucher, comme la forme, la taille, le poids, ou la température des objets.
- La Fonction sémiotique : La main joue le rôle d'organe d'expression pour l'émission d'informations visuelles. Cela comprend la langue des signes, les gestes qui accompagnent la parole, ou ceux utilisés dans des situations où l'on ne peut pas parler, par exemple en milieu bruyant ou sous l'eau. [1]

Dans le cadre des interfaces homme-machine, nous nous concentrons surtout sur la fonction sémiotique, qui est la plus complexe et variée, et qui peut être classée de différentes façons.

1.2.2 Types de Gestes

Selon l'évolution temporelle de la forme des gestes, on peut distinguer deux types de gestes, les gestes statiques, et les gestes dynamiques.

- **Geste statique** : L'information est transmise par une seule posture de la main, sans mouvement.
- **Geste dynamique** : Contrairement au geste statique, ce type de geste transmet l'information à travers la position de la main et son mouvement dans le temps. C'est le type de geste le plus difficile à analyser. [2]

Figure 1.01 **:** Geste Statique

Figure 1.02 **:** Geste dynamique

1.2.3 Dispositifs d'interaction

Aujourd'hui, quand on utilise un ordinateur, on se sert surtout de la souris pour cliquer sur des icônes, ouvrir des fenêtres ou choisir dans des menus. C'est ce qu'on appelle le système WIMP (Window, Icon, Menu, Pointing device en anglais) ou "Fenêtre, Icône, Menu, Pointers". En plus de ces méthodes classiques, il existe des interfaces haptiques, qui fournissent un retour tactile à l'utilisateur,

Dans cette section, nous examinerons les différents dispositifs d'interaction avec l'ordinateur, comme les périphériques d'entrée (souris et autres dispositifs de pointage), les gants de données, les caméras qui détectent les mouvements, et les écrans tactiles.

Figure 1.03 **:** Différents types de dispositifs d'interaction et acquisition du geste.

1.2.3.1 Périphériques d'entrée

Pour interagir avec un ordinateur, la souris est devenue le périphérique d'entrée incontournable, mais il existe aussi des alternatives comme le trackball et le pavé tactile (touchpad) sur les ordinateurs portables. [3].

1.2.3.2 Gants électroniques

Les gants de données, également appelés gants électroniques ou gants numériques, sont des dispositifs équipés de capteurs capables de capter et fournir des informations précises sur la position de la main et les angles des articulations des doigts. Ils ont été largement utilisés dans le domaine de la reconnaissance de la langue des signes, en raison de leur capacité à fournir des mesures précises et fiables des mouvements des doigts et des mains. Cependant, malgré leur utilité, ces gants présentent des inconvénients : leur coût élevé et leur encombrement rendent leur utilisation contraignante.[3]

1.2.3.3 Ecrans tactile

Un écran tactile est un dispositif qui combine les fonctions d'affichage d'un écran et celles d'un dispositif de pointage, comme une souris ou un stylet. Cette technologie permet de simplifier l'interaction avec les appareils en réduisant le nombre de périphériques nécessaires. Les écrans tactiles sont présents dans de nombreux appareils, tels que les smartphones, les tablettes, les ordinateurs portables et les bornes interactives. Les modèles récents peuvent détecter plusieurs points de contact simultanément (multi-touch) et réagir à différents niveaux de pression.

1.2.3.4 Caméras Vidéo

Contrairement aux autres systèmes, les caméras vidéo détectent les mouvements de la main sans nécessiter d'équipement spécial ou de périphérique dédié. Cependant, on utilise quand même des gants colorés ou des petits autocollants pour aider l'ordinateur à mieux voir nos doigts. L'un des défis majeurs de cette approche est de développer des traitements efficaces pour analyser le flux vidéo et extraire des informations utiles à partir des nombreuses données captées. De plus, une caméra ne fournit que des informations en 2D. Pour obtenir des données en 3D, il est nécessaire d'utiliser deux caméras ou plus, ou d'appliquer des techniques de modélisation 3D. Cela pose un problème d'occultation, car certaines parties de la main peuvent être cachées dans l'image.

Aujourd'hui, avec l'augmentation de la puissance des calculs des ordinateurs, l’intelligence artificielle et la disponibilité de caméras abordables, il est désormais possible de créer des systèmes de reconnaissance gestuelle en temps réel. Dans le cadre de ce mémoire, nous avons exploré cette problématique. Les caractéristiques des caméras, telles que le taux de rafraîchissement et la résolution, sont importantes : des valeurs élevées permettent des images plus détaillées et plus fréquemment mises à jour, mais augmentent aussi la complexité et le temps de traitement.

1.3 La Vision artificielle

1.3.1 Définition

La vision artificielle, également appelée vision par ordinateur ou vision numérique est une branche de l'intelligence artificielle dédiée à l'analyse, au traitement et à la compréhension des images et des vidéos. Elle fonctionne en capturant des données visuelles à l'aide de caméras et de capteurs, puis en les analysant grâce à des algorithmes avancés.

Cette technologie vise à reproduire certaines capacités de la vision humaine, permettant ainsi aux systèmes informatiques d'identifier, de classer et de prendre des décisions basées sur les informations visuelles qu'ils reçoivent. Elle est largement utilisée dans des domaines tels que la robotique, la reconnaissance de gestes, la sécurité, la réalité augmentée, et les véhicules autonomes. Le but ultime de la vision artificielle est de permettre aux ordinateurs de percevoir et de comprendre l'environnement de manière similaire à un être humain. [4]

1.3.2 Mécanisme de la Vision Humaine

La vision humaine est un processus complexe qui nous permet de percevoir les couleurs, les formes et les distances, et de comprendre notre environnement. Essentielle pour interagir avec le monde, elle nous aide à observer, lire, travailler, créer...

Le mécanisme de la vision peut être comparé à celui d'un appareil photo : la pupille contrôle la quantité de lumière qui entre, tandis que le cristallin ajuste la mise au point, comme un objectif. Les images sont ensuite projetées sur la rétine, à l'arrière de l'œil, qui convertit la lumière en signaux électriques transmis au cerveau. C'est ce dernier qui transforme ces signaux en images, rendant ainsi la vision possible. [5]

1.3.3 Perception du couleur

La vision dépend de la lumière, qui est à l'origine des couleurs. Les objets n'ont pas de couleur en eux-mêmes, c'est la lumière qu'ils reflètent qui crée cette impression. Ils absorbent certaines radiations et en renvoient d'autres, ce qui produit la perception de la couleur. En réalité, la couleur est une sensation physiologique que notre cerveau interprète à partir des signaux reçus par nos yeux. [6]

1.3.3.1 Lumière.

La lumière englobe une gamme d'énergie électromagnétique qui se compose de différentes longueurs d'onde (ou fréquences). Ces longueurs d'onde, dans le spectre visible, vont d'environ 380 à 720 nm. En bref, la lumière est de l'énergie émise à certaines fréquences, avec une intensité spécifique. [5]

1.3.3.2 Couleur

La couleur de la lumière dépend de sa fréquence, laquelle est directement liée à sa longueur d'onde, comme l'exprime la relation (1.01)

$$\lambda = c \times T \qquad (1.01)$$

- λ : Longueur d'onde
- c : Célérité de la lumière (3×10^{8} m.s^{-1})
- T : Période de l'onde

1.3.4 Principe de la vision artificielle

La vision artificielle cherche à reproduire la vision humaine à l'aide de technologies informatiques pour permettre aux machines de percevoir et comprendre leur environnement. Elle repose sur l'utilisation de différentes sources de rayonnement, telles que la lumière visible ou l'infrarouge, pour éclairer les objets ou les scènes. Un système d'imagerie capture ce rayonnement et crée une image à l'aide de lentilles et autres composants optiques.

Les capteurs d'image, transforment cette image en signaux électriques que l'ordinateur peut traiter. Les algorithmes de traitement d'image améliorent la qualité de l'image, segmentent les objets, extraient des caractéristiques et reconnaissent des objets. Enfin, l'interprétation des données permet à la machine de prendre des décisions ou d'effectuer des actions basées sur les informations visuelles reçues. En somme, la vision artificielle intègre des principes d'optique, d'électronique et d'intelligence artificielle pour doter les machines de capacités visuelles comparables à celles des humains.[7]

1.4 Traitement d'image

1.4.1 Introduction

L'image, tout comme la parole, est un moyen de communication puissant et universel. Elle permet aux gens de tous âges et cultures de se comprendre, chacun l'interprétant à sa façon. De ce fait, Le traitement d'images (en anglais, *Image Processing*) regroupe l'ensemble des méthodes et techniques utilisées pour améliorer une image ou en extraire des informations utiles. Son objectif est de rendre l'image plus facile à analyser, plus claire et plus agréable à regarder. Ces techniques, principalement appliquées aux images numériques, sont étroitement liées aux sciences informatiques et au traitement du signal. Des exemples classiques incluent l'analyse des textures d'images (comme la distribution des pixels), le filtrage (par des méthodes

comme les filtres morphologiques ou les détecteurs d'angles), ainsi que la transformation d'images (comme l'extraction de caractéristiques ou la fusion d'images).

1.4.2 Notion d'image

Une image est une représentation d'une personne ou d'un objet par la peinture, la sculpture, le dessin, la photographie, le film, etc. C'est aussi un ensemble structuré d'informations qui, après affichage sur l'écran, ont une signification pour l'œil humain.

Une image peut être décrite comme une fonction continue de luminosité I(x, y), où x et y sont les coordonnées d'un point de l'image, et I représente l'intensité lumineuse et la couleur. Cependant, sous cette forme, l'image ne peut pas être traitée par une machine, ce qui nécessite de la numérisation. [8]

1.4.3 Image numérique

Les images manipulées par un ordinateur sont numériques, représentées par une série de bits, contrairement aux photos classiques ou dessins sur papier. Une image numérique se compose de cellules appelées pixels, qui couvrent toute la surface de l'image à tailles fixes.

Chaque pixel correspond à un niveau de gris ou de couleur, déterminé soit par l'emplacement dans l'image réelle, soit par un calcul basé sur une description interne de la scène représentée.

La numérisation d'une image est la conversion de celle-ci de son état analogique (distribution continue d'intensités lumineuses dans un plan $\overrightarrow{xoy}$ .En une image numérique représentée par une matrice bidimensionnelle de valeurs numériques *f(x,y)* où :

$$\begin{cases} x, y : \text{coordonnées cartésiennes d'un point de l'image} \\ f(x, y) : \text{niveau de gris en ce point} \end{cases}$$

Pour faciliter l'affichage et la gestion, les données d'image sont organisées en un tableau de n lignes et p colonnes. Chaque élément du tableau, noté *I(x ,y)* représente un pixel de l'image et à sa valeur est associé un niveau de gris codé sur m bits (2^m niveaux de gris ; 0 = noir ; 2^m-1 = blanc). La valeur en chaque point exprime la mesure d'intensité lumineuse perçue par le capteur. [8]

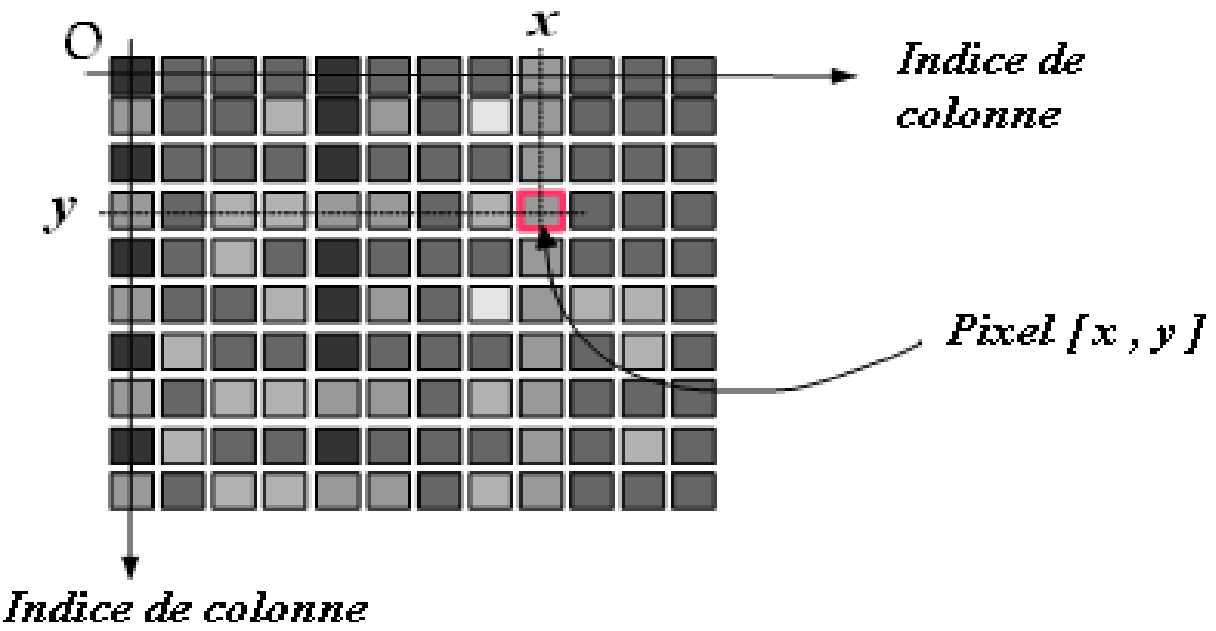


Figure 1.04 : Image numérique

1.4.4 Caractéristiques d'une image numérique

L'image est un ensemble structuré d'informations caractérisé par les paramètres suivants :

1.4.4.1 Pixel

Le pixel, qui vient de l'expression anglaise "picture elements", est le plus petit point d'une image. C'est une unité de base pour les systèmes d'affichage et d'impression. Chaque pixel contient de l'information : dans une image en noir et blanc ou une image monochrome, il est codé sur un octet. Pour une image en couleur, un pixel utilise trois octets, un pour chaque couleur primaire RVB (rouge, vert, bleu). La quantité de mémoire nécessaire pour afficher une image dépend donc de la taille de l'image et du nombre de couleurs. [8]

1.4.4.2 Dimension

C'est la taille de l'image, ou encore le nombre total de pixels dans l'image. Cette dernière se présente sous forme de matrice dont les éléments sont des valeurs numériques représentatives des intensités lumineuses (pixels). Donc la dimension D d'une image est le nombre de ligne M multiplié par le nombre de colonne N de cette matrice. [6]

1.4.4.3 Résolution

C'est la clarté ou le niveau de détail d'une image sur un moniteur ou une imprimante. Elle se mesure en pixels par unité (pouce ou centimètre). La résolution peut aussi désigner le nombre total de pixels affichés horizontalement et verticalement : plus ce nombre est élevé, plus l'image est nette.

1.4.4.4 Bruit

Un bruit (ou parasite) dans une image est considéré comme un phénomène de brusque variation de l'intensité d'un pixel par rapport à ses voisins, il provient de l'éclairage des dispositifs optiques et électroniques du capteur**.**

1.4.4.5 Histogramme

L'histogramme des niveaux de gris ou des couleurs d'une image est une fonction qui donne la fréquence d'apparition de chaque niveau de gris (couleur) dans l'image. Pour diminuer l'erreur de quantification, pour comparer deux images obtenues sous des éclairages différents, ou encore pour mesurer certaines propriétés sur une image, on modifie souvent l'histogramme correspondant**.** [8]

1.4.4.6 Contours et textures

Les contours représentent la frontière entre les objets de l'image, ou la limite entre deux pixels dont les niveaux de gris représentent une différence significative. Les textures décrivent la structure de ceux-ci. L'extraction de contour consiste à identifier dans l'image les points qui séparent deux textures différentes**. [8]**

1.4.5 Les systèmes de traitement d'images

Un système de traitement d'images est représenté sur la figure suivante :

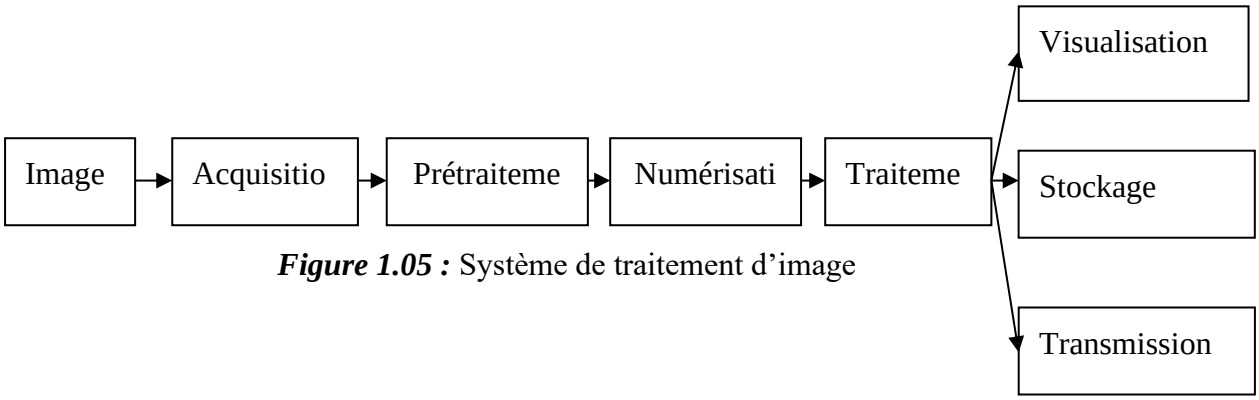


Figure 1.05 : Système de traitement d'image

1.4.5.1 Acquisition des données images

L'acquisition d'images est une étape clé dans la création et la manipulation d'images numériques. Pour qu'une image soit lisible sur un ordinateur, elle doit être numérisée, c'est-à-dire convertie en un format que le système peut traiter. Ce processus est effectué par des appareils comme les caméras numériques et les scanners. Ces technologies, notamment les

scanners, sont largement utilisées dans des domaines comme la télédétection, la médecine et les arts graphiques. De plus, des dispositifs comme les cartes d'acquisition, équipés de capteurs CCD, capturent des images provenant de caméras ou de télévisions pour les convertir en données numériques. [8]

1.4.5.2 Prétraitement

Le prétraitement des images, souvent via un filtre passe-bas, est utilisé pour corriger les dégradations subies lors de l'acquisition. Ces dégradations proviennent de phénomènes physiques comme la diffraction (déviation) du système optique et le flou causé par le mouvement pendant la capture. Le filtrage est généralement utilisé pour atténuer ces effets.

1.4.5.3 Numérisation

Suivant l'objet ou le document à numériser et le domaine d'application dans lequel l'image va être utilisée, il existe divers dispositifs de numérisation d'images allant du simple scanner à main au satellite de télédétection.

1.4.5.4 Les différents traitements

Il existe plusieurs traitements que l'on peut appliquer à une image selon le besoin ou le but à atteindre. Parmi lesquels :

- La manipulation de contraste
- L'opération de filtrage
- La compression
- Le tatouage numérique
- La segmentation d'image

1.4.5.5 Utilisation du traitement d'images

Le traitement d'image est utilisé pour la visualisation, le stockage et la transmission des images.

1.4.6 Les techniques segmentation d'image pour la reconnaissance des gestes

La segmentation est une étape clé dans la reconnaissance des gestes de la main. Il existe plusieurs méthodes pour segmenter l'image, qui varient en fonction des conditions de la scène (comme l'éclairage ou le lieu, intérieur ou extérieur). Notre objectif est non seulement de détecter la main, mais aussi de bien en définir le contour pour en extraire des informations sur sa forme et sa position. Comme la segmentation n'est que la première étape d'un système de

reconnaissance de gestes, il est essentiel de trouver un équilibre entre la précision et la rapidité, car le tout doit fonctionner en temps réel. [9]

1.4.6.1 Definition

La segmentation d'images consiste à diviser une image en plusieurs régions homogènes en fonction des propriétés communes des pixels, comme la couleur, la texture ou l'intensité. Zucker [20] définit la segmentation comme le fait de découper une image I en n sous-ensembles distincts, appelés régions Ri, où chaque région regroupe des pixels aux caractéristiques similaires. L'image I est donc l'ensemble de ces régions, c'est-à-dire $I = R1 \cup R2 \cup ... \cup Rn$, où chaque région est différente des autres. L'objectif de cette segmentation est de simplifier l'image pour faciliter son analyse, par exemple dans des applications comme la reconnaissance d'objets ou de formes. [9]

1.4.6.2 Différentes approches de segmentation d'images :

La segmentation d'image est un domaine vaste où l'on trouve de très nombreuses approches. Toutes ces approches visent à extraire des indices visuels et cela de différentes façons (Figure 1.05**).**

- Approche par contours :
  Cette approche manipule des zones entières de l'image. Certaines méthodes commencent par une première division de l'image, qui est ensuite affinée en divisant ou regroupant les régions (méthodes de division/fusion). D'autres méthodes partent de quelques régions initiales qui s'étendent en incorporant des pixels jusqu'à couvrir toute l'image, ce qui correspond aux méthodes de croissance de régions.[9]
- Approche par régions :
  Cette approche manipule des zones entières de l'image. Certaines méthodes commencent par une première division de l'image, qui est ensuite affinée en divisant ou regroupant les régions (méthodes de division/fusion). D'autres méthodes partent de quelques régions initiales qui s'étendent en incorporant des pixels jusqu'à couvrir toute l'image, ce qui correspond aux méthodes de croissance de régions.
- Approche par seuillage :
  Le seuillage est une technique de segmentation très populaire en raison de sa simplicité et de sa rapidité. Elle permet de séparer les objets du fond de l'image. En général, les pixels sont classés en deux groupes selon un seuil de gris S : ceux qui appartiennent au fond et ceux qui font partie de l'objet. Ce processus est appelé binarisation. [9]

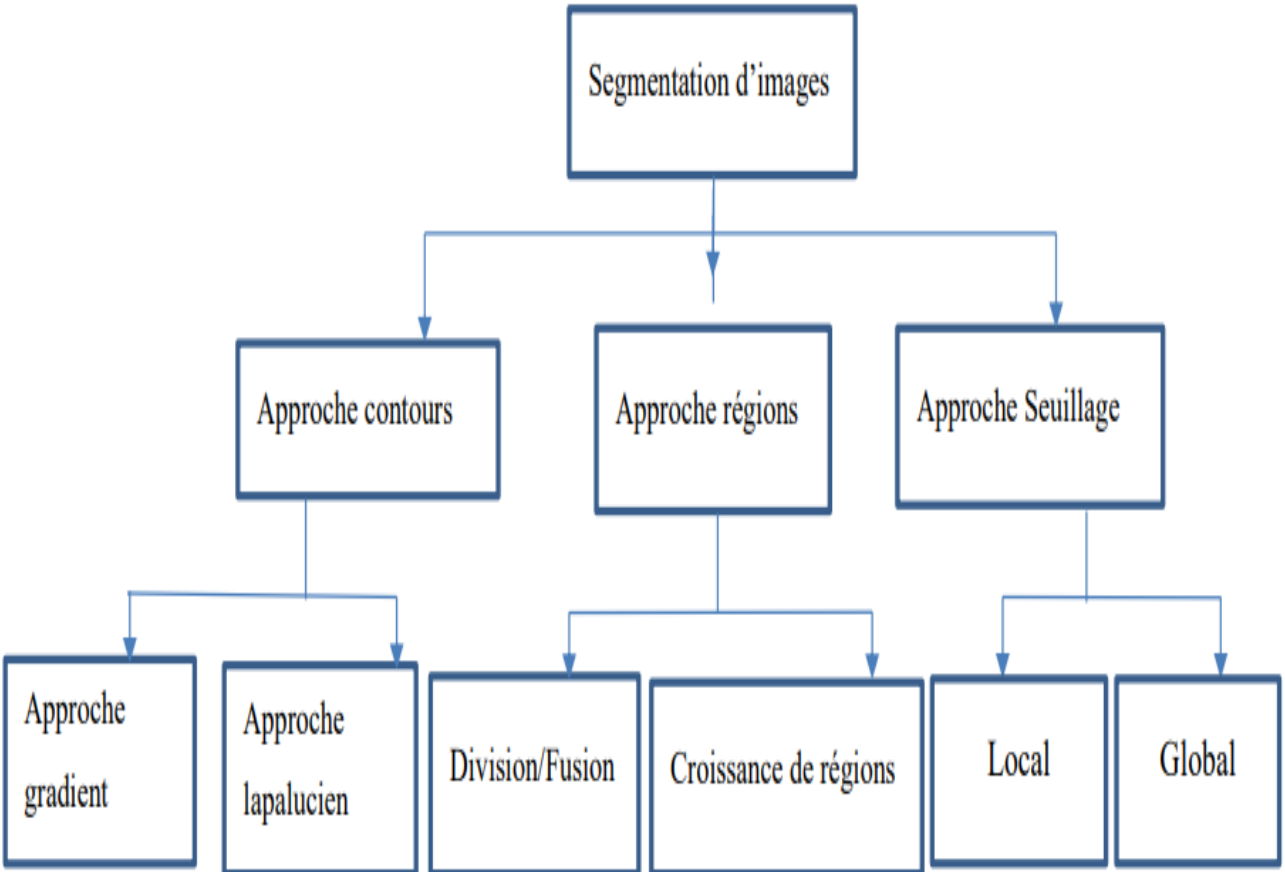


Figure 1.06 **:** Approches de segmentation d'images

Conclusion

Dans ce premier chapitre, nous avons exploré les bases des gestes et les principes de la vision par ordinateur, qui cherche à imiter le système visuel humain. Cette introduction nous a permis de développer des compétences solides en traitement d'images et de comprendre les principes fondamentaux de la reconnaissance de gestes.

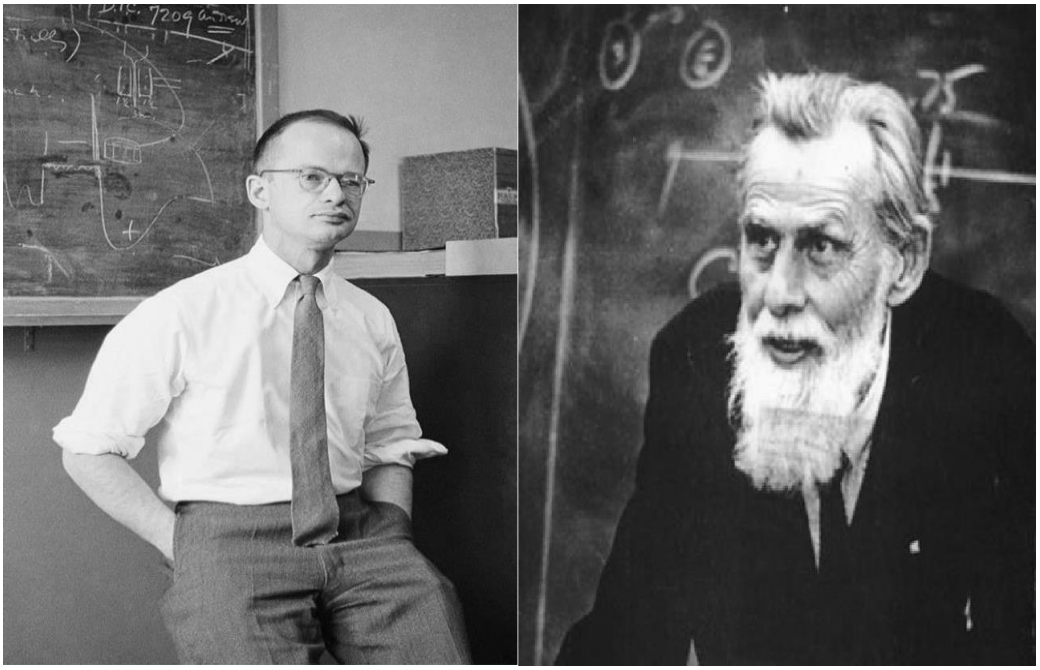
CHAPITRE 2 :
RÉSEAU NEURONAL POUR LA RECONNAISSANCE GESTUELLE

2.1 Introduction

La reconnaissance des gestes est un domaine clé dans le développement d'interfaces homme-machine intuitives. Pour réaliser cette tâche, nous faisons appel à des réseaux neuronaux, une technique d'apprentissage profond (ou Deep Learning en anglais) issue de l'apprentissage automatique (Machine Learning en anglais). L'apprentissage automatique est un sous-domaine de l'intelligence artificielle qui permet aux machines d'apprendre à partir d'exemples et de données. En intégrant ces données dans un modèle, la machine peut effectuer des prédictions ou des classifications. Les réseaux neuronaux, avec leur architecture multicouche, extraient des caractéristiques complexes pour apprendre des tâches sophistiquées comme la reconnaissance de gestes ou l'identification d'objets. [14]

2.2 Historique

Les débuts de l'intelligence artificielle remontent à 1943, quand Warren McCulloch et Walter Pitts, un duo de chercheurs alliant mathématiques et neurosciences, ont posé les bases des réseaux de neurones. Dans leur étude novatrice, ils ont proposé un modèle inspiré du cerveau humain pour créer des neurones artificiels. Leur article, « A logical calculus of the ideas immanent in nervous activity », a marqué le début de l'IA moderne.[15]



Warren MaCulloch Walter Pitts

Figure 2.01: Walter Pitts et Warren MaCulloch

Environ quinze ans plus tard, en 1957, l'histoire de l'intelligence artificielle connut une avancée majeure. Frank Rosenblatt, un psychologue américain, franchit une étape cruciale en développant le Perceptron. Cette innovation représente le premier algorithme d'apprentissage dans l'histoire de l'apprentissage profond. Le Perceptron de Rosenblatt a permis d'améliorer significativement le modèle initial, ouvrant la voie à des systèmes d'IA capables d'apprendre par eux-mêmes.

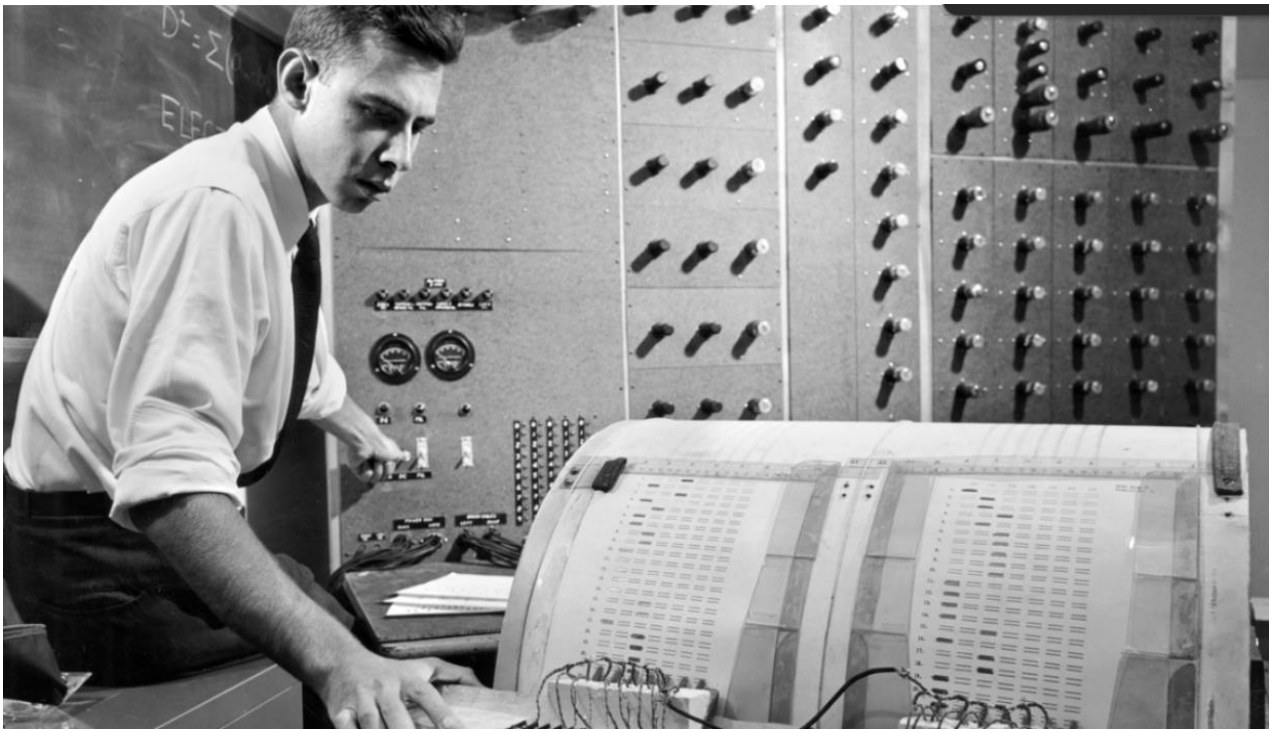


Figure 2.02 : Frank Rosenblatt

Entre 1974 et 1980, l'intelligence artificielle a connu une période de stagnation, souvent appelée "le premier hiver de l'IA", marquée par un manque de financement et un intérêt décroissant. Cependant, le vent a tourné en 1986. Geoffrey Hinton, aujourd'hui considéré comme un pionnier du Deep Learning, a ravivé l'intérêt pour l'intelligence artificielle en développant le perceptron multicouche, le premier véritable réseau de neurones artificiels. Cette innovation a donné un nouvel élan au domaine. Au fil du temps, ce modèle s'est amélioré avec l'ajout de nouvelles fonctions d'activation, comme la fonction logistique, la tangente hyperbolique et la fonction ReLU, qui ont renforcé l'efficacité des réseaux de neurones.[15]

Figure 2.03 : Geoffrey Hinton

Dans les années 1990, Yann LeCun a apporté une avancée clé avec l'invention des réseaux de neurones convolutifs, qui ont transformé le traitement d'images par l'intelligence artificielle. Ces réseaux, inspirés du fonctionnement du cerveau humain, utilisent des opérations mathématiques appelées « convolutions et pooling » pour analyser une image par petites sections avant de la traiter. Cette méthode a amélioré la capacité des machines à reconnaître et comprendre les images, ouvrant ainsi la voie à de nombreuses applications en vision par ordinateur. [16]

Figure 2.04 : Yan LeCun

En 1997, les réseaux de neurones récurrents ont amélioré le traitement des données séquentielles comme le texte et la reconnaissance vocale. En 2012, Geoffrey Hinton et son

équipe ont marqué un tournant pour l'IA en développant un réseau surpassant tous les algorithmes en reconnaissance d'images.[16]

2.3 Réseaux de neurone Artificiel

2.3.1 Neurone biologique

Les réseaux de neurones artificiels ou RNN sont des **modèles mathématiques** inspirés de la biologie. La brique de base de ces réseaux, le neurone artificiel, était issu au départ d'une volonté de modélisation du fonctionnement d'un neurone biologique. Jetons maintenant un coup d'œil à la figure 2.05 pour comprendre comment fonctionne un neurone. [10]

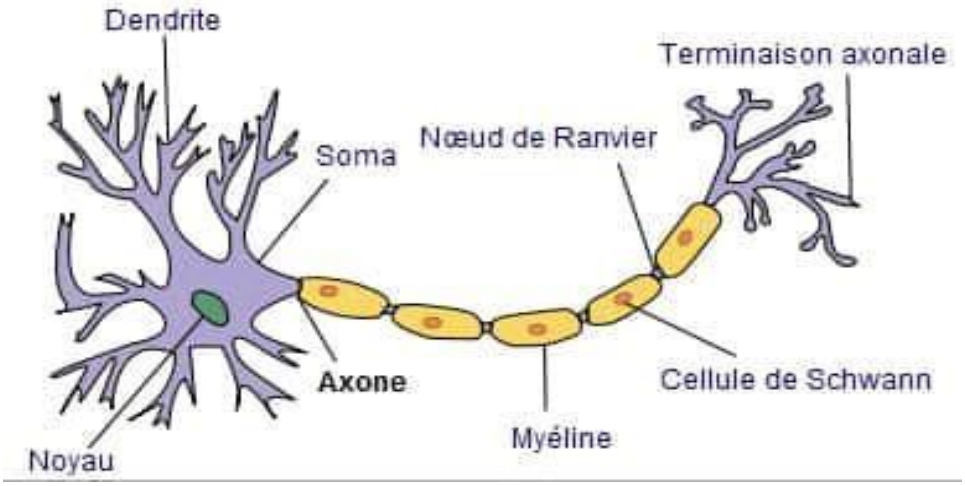


***Figure 2.05* :** Cellule Neurone

Schématiquement, Un neurone peut être divisé en trois parties principales :

- Un **corps cellulaire**, appelé *péricaryon* ;
- Un **ensemble de dendrites** (de l'ordre de 7 000) ;
- Un **axone**.

Les dendrites sont les capteurs du neurone. Ils reçoivent des informations sous forme de signaux nerveux provenant de stimuli, qu'elles transmettent ensuite au corps cellulaire. On dit que le flux est *centripète*. [11]

2.3.2 Le Neurone Formel

Le **perceptron**, aussi appelé **neurone artificiel** ou **neurone formel** (voir Figure 2.06), cherche à imiter le fonctionnement d'un neurone biologique. Il est à la base de l'apprentissage profond et fonctionne en trois étapes : [12]

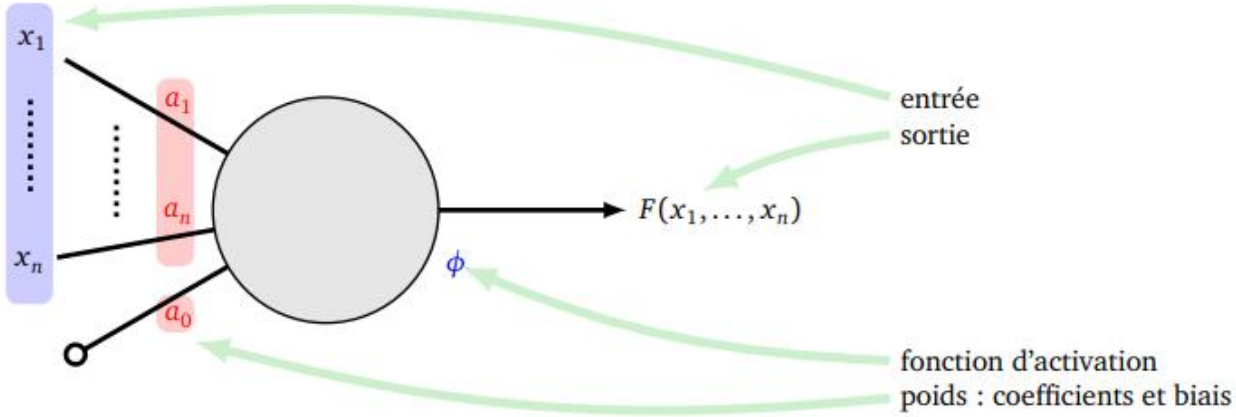


Figure 2.06 : Modélisation de réseau de neurone (Neurone Formel)

Les entrées (x) :

Ce sont les données reçues par le perceptron, représentées sous forme de vecteurs $\boldsymbol{x}$, similaires aux dendrites dans un neurone biologique. Chaque entrée est pondérée par un coefficient $\boldsymbol{a}$ (appelé poids), qui détermine son importance. Il y a aussi une valeur spéciale appelée biais ($\boldsymbol{a_0}$), toujours égale à 1, qui permet au perceptron de mieux s'adapter et d'apprendre plus efficacement. [12]

Les Noyau :

C'est la partie centrale du perceptron, Il effectue la **somme pondérée** des entrées avec leurs poids respectifs.

$$a_1x_1 + \ldots + a_nx_n = \sum_{i=1}^{n} a_ix_i \qquad (2.01)$$

La sortie :

C'est la valeur finale du perceptron, obtenue en passant la valeur finale du noyau à travers une **fonction d'activation (ϕ)**, souvent non linéaire, généralement les plus utiliser sont :

- Fonction marche de Heaviside :

$$H(x) = \begin{cases} 0 & si\ x<0 \\ 1 & si\ x\geq 0 \end{cases} \qquad (2.02)$$

- Unité de rectification linéaire (ReLU) :

$$ReLU(x) = \begin{cases} 0 & si\ x<0 \\ x & si\ x\geq 0 \end{cases} \qquad (2.03)$$

- Fonction sigmoïde :

$$\sigma = \frac{1}{1+e^{-x}} \qquad (2.01)$$

- Fonction tangente hyperbolique :

$$\tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}} \tag{2.05}$$

Ces exemples de fonctions d'activations ont en points communs de ne pas être des polynômes afin de ne pas ralentir le calcul mais aussi d'être dérivables, indispensable pour calculer le gradient de l'erreur commise par le neurone et ainsi permettre son apprentissage.

La fonction de sortie est donnée par :

$$f = \phi(\langle a, x \rangle + a_0) \tag{2.06}$$

Cette équation dit comment la sortie est calculée. Chaque entrée est multipliée par un poids, un coefficient a. Toutes les entrées sont alors sommées et additionnées à un biais $\boldsymbol{a_0}$. Le résultat de la somme passe à travers une fonction de transfert. Cette fonction produit alors la sortie voulue.

On le voit, plus les entrées sont grandes en valeur absolue, plus la somme peut être grande en valeur absolue. Nous sommes donc en **modulation d'amplitude** (et non plus de fréquence, comme dans un neurone biologique).[11] [12]

2.3.3 Théorie du perceptron (OU, ET, OU exclusif)

Le perceptron est la brique de base des réseaux de neurones. C'est un modèle de classification binaire qui permet de séparer deux groupes de données à l'aide d'une ligne. [13]

- **OU :**

C'est une opération entre deux valeurs booléennes. Un booléen est une variable qui peut prendre deux valeurs : "vrai" (1) ou "faux" (0). Avec deux booléens, x et y, l'opération "x OU y" est "vraie" si l'un des deux (ou les deux) est vrai.

Graphiquement, On peut illustrer toutes les combinaisons possibles de "x OU y" à l'aide d'un diagramme. Dans ce diagramme, un rond bleu en position (x, y) signifie que "x OU y" est faux (0), et un carré rouge indique que "x OU y" est vrai (1). [13]

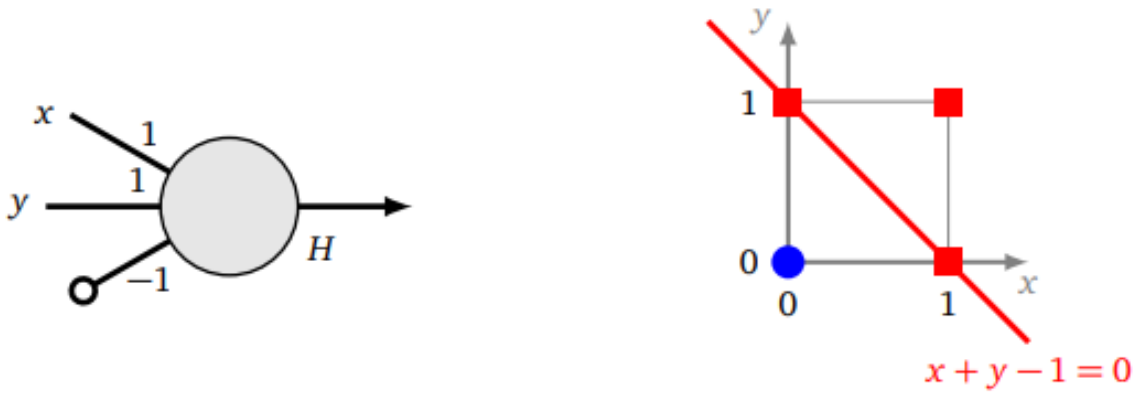


Figure 2.07 : Opération OU (Neurone Formel)

On peut effectivement réaliser l'opération "x OU y" à l'aide d'un perceptron. Cela revient à séparer les ronds bleus (faux) des carrés rouges (vrai). Un perceptron avec des poids $a = 1$, $b = 1$, et $c = -1$ permet de faire cette séparation.

- Demonstration : $\boldsymbol{f(x,y) = x + y - 1}$ **avec** $\boldsymbol{a = 1}$, $\boldsymbol{b = 1}$, et $\boldsymbol{c = -1}$

En utilisant la fonction de Heaviside, on a

$$f(x) = \begin{cases} 0 & si\ x<0 \\ 1 & si\ x\geq 0 \end{cases}$$

Par exemple, si $\boldsymbol{x = 0}$, $\boldsymbol{y = 1}$ et Si $\boldsymbol{x = 0}$ **et** $\boldsymbol{y = 0}$, alors

$1(0) + 1\,(1) - 1 = 0 \geqslant 0$, le perceptron renvoie 1 ("vrai")

$1\,(0) + 1(\,0\,) - 1 = -1 < 0$, alors le perceptron renvoie 0 ("faux").

Ainsi, le perceptron parvient à modéliser l'opération "OU".

- **ET(AND) :**

L'opération "x ET y" peut être réalisée par un perceptron en utilisant les poids $a = 1$, $b = 1$ et $c = -2$. Le perceptron renvoie "vrai" uniquement lorsque les deux entrées sont vraies. Par exemple, si $\boldsymbol{x = 1}$ et $\boldsymbol{y = 1}$,

$1(1) + (1) - 2 = 0$, donc le perceptron renvoie 1 ("vrai").

Si une ou les deux entrées sont fausses, le résultat sera inférieur à 0, donc le perceptron renverra 0 ("faux").[13]

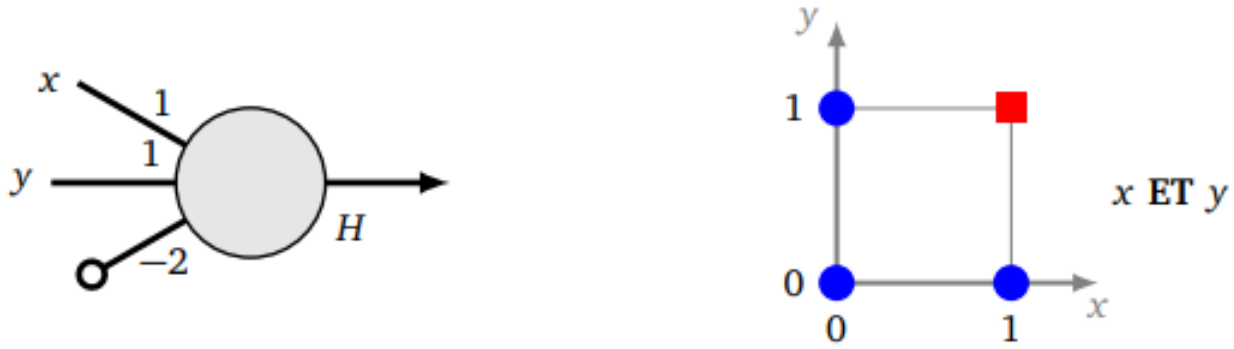


Figure 2.08 : Opération ET (Neurone Formel)

- **OU Exclusif (XOR) :**

L'OU exclusif, noté « x XOR y », est vrai lorsque x ou y est vrai, mais pas les deux en même temps. Contrairement à l'OU classique, l'OU exclusif ne peut pas être réalisé par un perceptron. En effet, il n'existe pas de perceptron (à deux entrées, avec une fonction d'activation de type Heaviside) capable de modéliser cette opération. Géométriquement, cela

signifie qu'il n'existe pas de ligne droite qui puisse séparer les combinaisons où "x XOR y" est vrai de celles où il est faux.

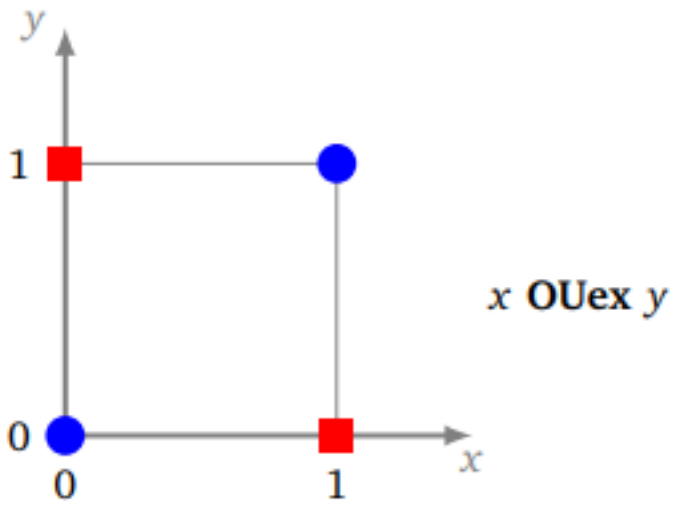


Figure 2.09 **:** Représentation des données OU Exclusif (XOR)

Après avoir vu comment un neurone formel reçoit des signaux et produit une sortie, la question est : comment trouver les paramètres pour rendre les prédictions précises ? Il faut trouver les bons poids pour que la sortie soit proche de la valeur attendue. C'est ici que l'apprentissage entre en jeu. Il vise à minimiser l'erreur entre la sortie prédite et la valeur réelle. [13]

2.3.4 Optimisation d'un Neurone Artificiel

La plupart des algorithmes en apprentissage artificiel nécessitent une optimisation, qui consiste à maximiser ou minimiser une fonction $f(x)$, appelée fonction coût, où x est la variable ici. Lorsqu'on cherche à la minimiser, elle est souvent appelée fonction d'erreur ou fonction de perte.

2.3.4.1 Fonction Coût

La fonction coût (ou fonction de perte) mesure l'erreur ou l'écart entre la prédiction faite par le modèle et la valeur réelle attendue. Son objectif est de quantifier la qualité de la prédiction. Une fonction coût typique pour un problème de régression est l'erreur quadratique moyenne (MSE - Mean Squared Error) mais elle peut dépendre du problème à résoudre (classification ou segmentation). Elle est définie comme suit :

$$MSE = J(\theta) = \frac{1}{n}\sum_{i=1}^{m}(y_i - S_i)^2 \qquad (2.07)$$

y_i : la valeur calculée (ou prédite) en sortie du neurone i,

S_i : la valeur attendue (ou réelle) en sortie du neurone i,

n : le **nombre total d'exemples** d'entraînement.

Cette fonction permet de voir comment l'ajustement des paramètres du modèle impacte l'erreur de prédiction. [12]

2.3.3.2 Descente de Gradient

La dérivée est utile pour minimiser une fonction parce qu'elle nous dit comment changer x pour faire une petite amélioration dans y, une méthode appelée « Gradient Descent ». C'est une méthode d'optimisation utilisée pour minimiser la fonction coût. Elle ajuste les paramètres du modèle (comme les poids des neurones) en suivant la direction opposée au gradient de la fonction coût par rapport à ces paramètres, dans le but de trouver un minimum global. L'algorithme de descente du gradient met à jour les paramètres θ selon la règle suivante :

$$\theta' = \theta - \alpha \frac{\partial J(\theta)}{\partial \theta} \tag{2.08}$$

θ' : le paramètre que l'on ajuste (valeur corrigée du poids) et θ la valeur actuelle du poids

α : Pas d'apprentissage, qui contrôle la vitesse de mise à jour des poids, $\frac{\partial J(\theta)}{\partial \theta}$: est le gradient de la fonction coût.

Le gradient indique dans quelle direction et avec quelle intensité les poids doivent être ajustés pour réduire l'erreur. [17]

2.3.4 Réseaux de neurone multicouche

Nous avons vu qu'un modèle avec un seul neurone permet de réaliser des tâches de classification simples, par exemple, en séparant les données en deux catégories par une ligne droite (lorsqu'il y a deux entrées) ou par une surface plane (lorsqu'il y a trois entrées ou plus). Cependant, un seul neurone n'est pas suffisant pour résoudre des problèmes plus complexes, comme l'opération "OU exclusif". Pour dépasser cette limite, on connecte plusieurs neurones ensemble, formant ce qu'on appelle un réseau de neurones multicouche. Cela permet de modéliser des relations plus complexes, un peu comme les neurones biologiques qui communiquent entre eux via des synapses pour traiter des informations plus compliquées. C'est ainsi qu'un réseau de neurones multicouches est formé. Ce type de réseau, aussi appelé Perceptron Multicouche (PMC) ou Multi-Layer Perceptron (MLP), est constitué de plusieurs neurones répartis en couches. L'information circule toujours de la couche d'entrée vers la couche de sortie. [11][13]

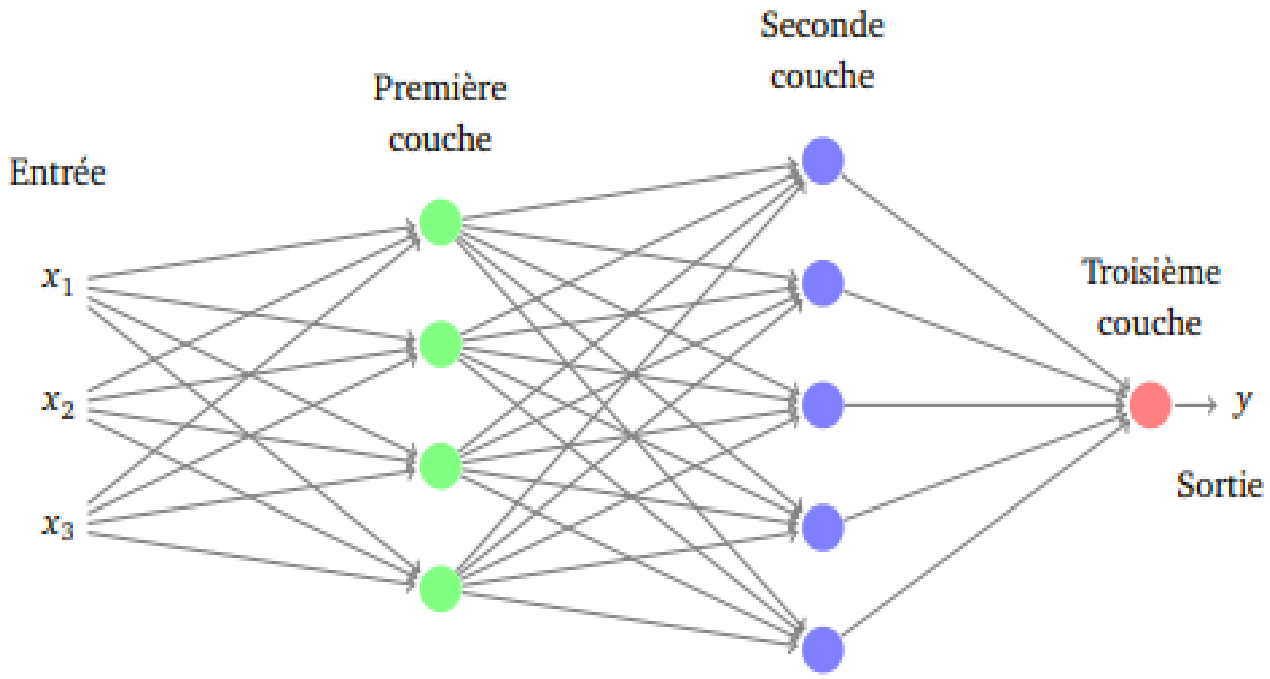


Figure 2.10 : Un réseau de neurones multicouches

Chaque réseau comporte une couche d'entrée, une ou plusieurs couches intermédiaires (ou couches cachées), et une couche de sortie. Chaque neurone dans le réseau est défini par des poids et une fonction d'activation. Généralement, on utilise la même fonction d'activation dans tout le réseau, sauf parfois pour la couche de sortie qui peut être différente.

Remarque :

Les Perceptrons Multicouches (PMC) sont bien adaptés aux données de taille fixe, comme les images. Cependant, pour les données de taille variable ou les séquences (comme le texte ou les séries temporelles), il est préférable d'utiliser des réseaux de neurones récurrents (RNN). Dans un réseau récurrent, certains neurones renvoient de l'information vers leur propre couche ou une couche précédente, ce qui permet de mieux traiter les données séquentielles.[11]

- Théorie des réseaux de neurones :

Nous avons vu qu'un seul neurone ne peut pas réaliser la fonction du « OU exclusif ». En revanche, avec plusieurs neurones, cela devient possible. Par exemple, un réseau de neurones peut diviser le plan en trois parties : une zone centrale où la fonction du réseau vaut 0, et une zone périphérique (y compris la frontière rouge) où la fonction vaut 1.

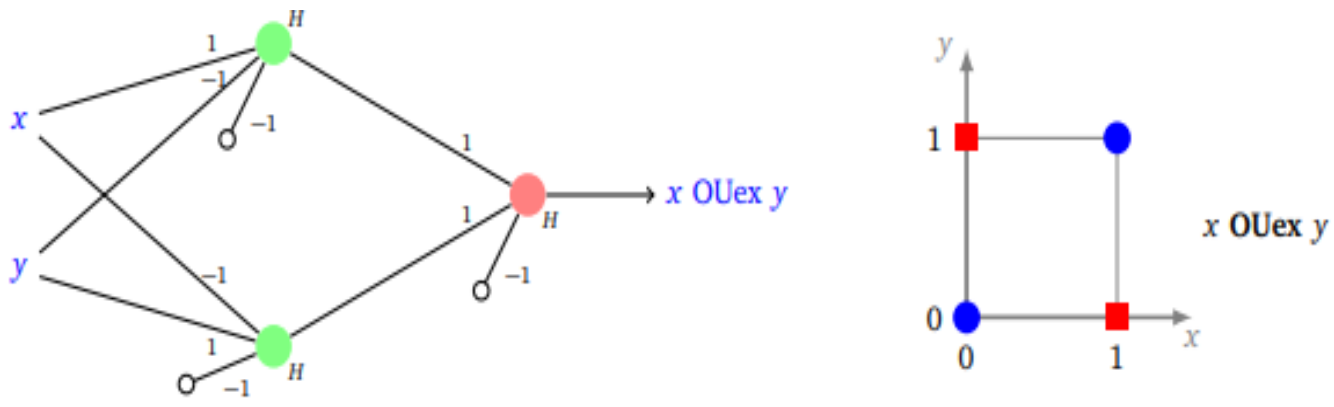


Figure 2.11 : Un réseau de neurones à une couche et 2 neurones

Ce réseau permet d'obtenir une valeur

$$f(x) = \begin{cases} 0 \ aux \ points \ (0,0) \text{ et } (1,1) \\ 1 \ aux \ points \ (1,0) \text{ et } (0,1) \end{cases}$$

L'idée derrière ce réseau repose sur le fait que l'opération « OU exclusif » est une combinaison de « OU » et de « ET » : x OUex y = (x ET non y) OU (non x ET y).

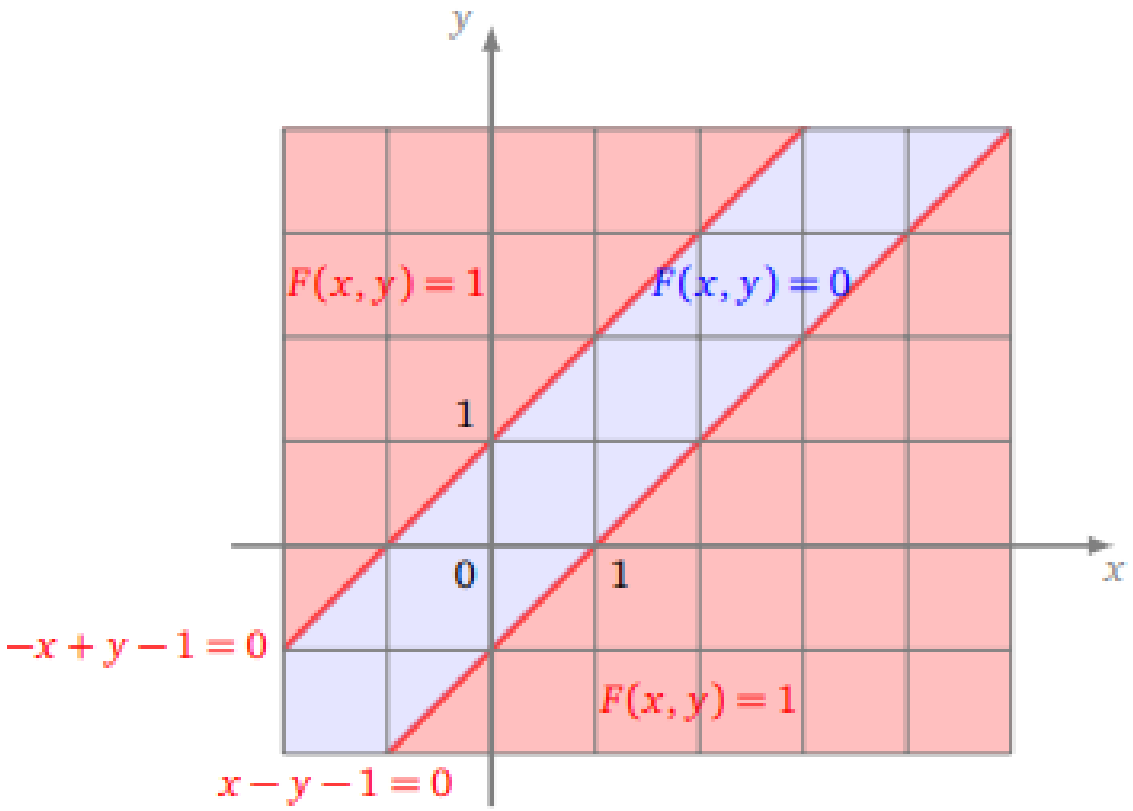


Figure 2.12 : Représentation des données OU Exclusif (XOR) Séparer par deux droites

2.4 Réseau de neurone convolutif

2.4.1 Architecture d'une CNN

Toujours basée sur le fonctionnement du neurone biologique, la nouvelle architecture de réseau neuronal convolutif (CNN), développé fin des années 90 par LeCun et al. Permet l'application de méthodes d'apprentissage profond sur des images notamment par l'extraction et la reconnaissance de motifs via des opérations de convolutions.[8] Leur architecture est plus spécifique : elle composée de deux blocs principaux.

- Une partie convolutive : Cette étape extrait les caractéristiques importantes de l'image en la réduisant. L'image passe à travers plusieurs filtres, générant des cartes de convolution. Ces cartes sont ensuite combinées en un vecteur appelé code CNN.
- Une partie classification : Le code CNN est ensuite utilisé par des couches entièrement connectées (perceptron multicouche) pour classer l'image.

L'architecture générale d'un CNN consiste en l'alternance des deux types de couches : la couche de convolution et la couche de pooling.

2.4.2 Couche convolution

La convolution est une opération mathématique simple couramment utilisée pour le traitement et la reconnaissance d'images. Elle effectue un filtrage sur l'image selon le processus suivant : Tout d'abord, une fenêtre de filtre est définie, généralement en haut à gauche de l'image. La fenêtre de filtre, qui représente la caractéristique recherchée, se déplace progressivement de gauche à droite sur un certain nombre de positions prédéfinies (appelées pas), jusqu'à atteindre la fin de l'image.

À chaque position de la fenêtre sur l'image, un calcul de convolution est effectué, ce qui permet d'obtenir une carte d'activation ou une carte des caractéristiques. Cette carte indique où les caractéristiques sont localisées dans l'image : plus la valeur dans la carte d'activation est élevée, plus la portion de l'image balayée ressemble à la caractéristique recherchée. [10]

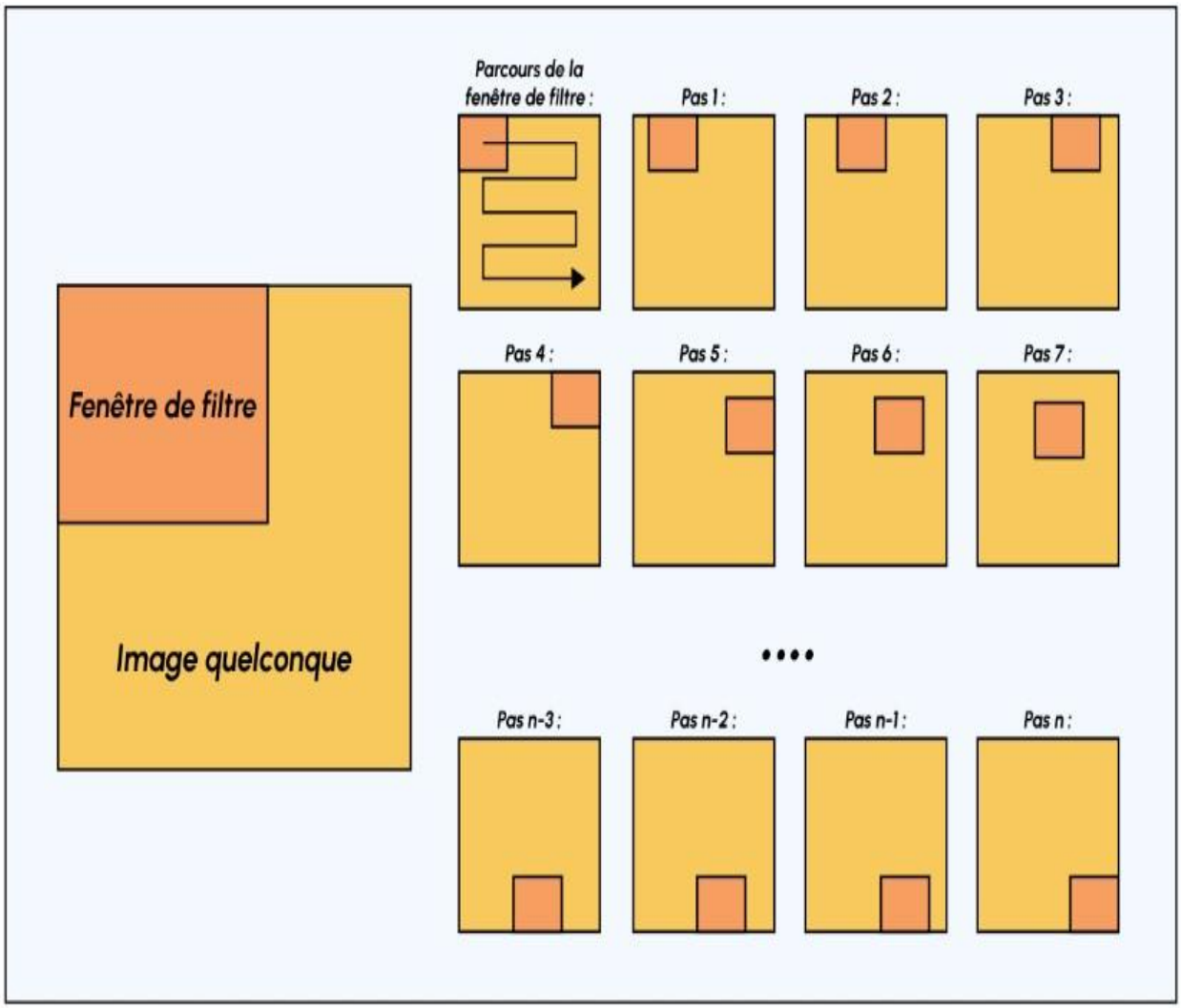


Figure 2.13 : Schéma du parcours de la fenêtre de filtre sur l'image

Dans les réseaux de neurones convolutifs (CNN), l'image passe à travers plusieurs filtres de convolution qui extraient des caractéristiques importantes, comme les contours ou les formes. Ces filtres, comme le filtre moyenneur ou le filtre gaussien, sont automatiquement ajustés par le modèle pour détecter des éléments spécifiques dans l'image. Voici un exemple illustrant les effets de ces deux filtres différents sur une image avec un fort bruit (comme une photographie prise dans des conditions de faible luminosité). Cependant, l'un des inconvénients de la réduction du bruit est qu'elle est souvent accompagnée d'une diminution de la netteté de l'image :

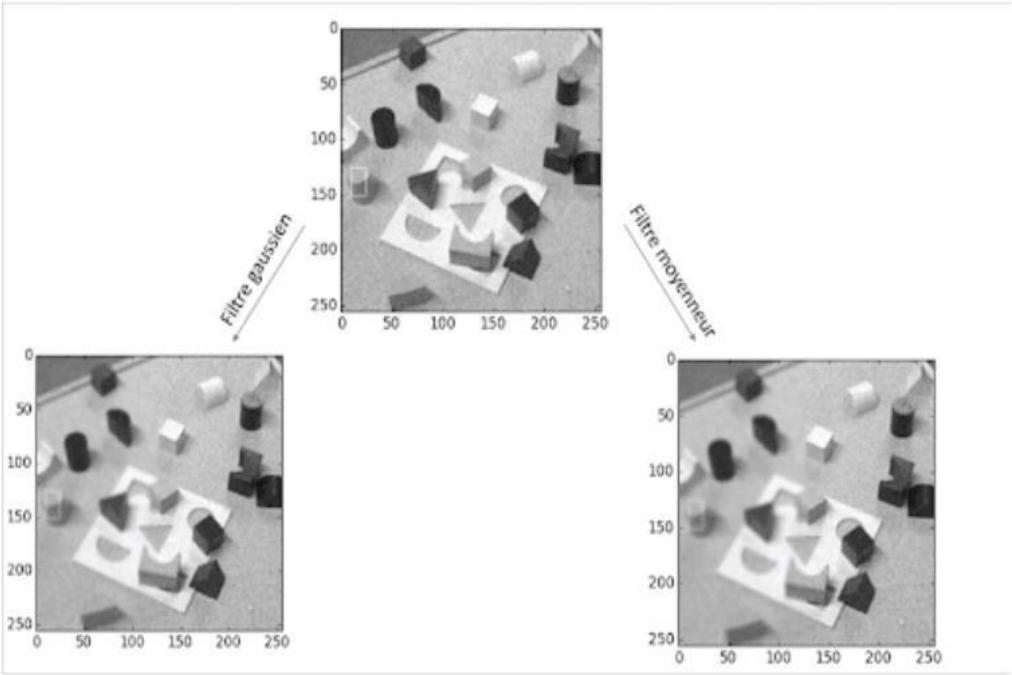


***Figure 2.14*:** Effet des filtres moyenneur et gaussien

Comme on peut le constater, contrairement au filtre moyenneur, le filtre gaussien parvient à réduire le bruit sans altérer significativement la netteté de l'image. [10]

2.4.3 Couche pooling

La couche de pooling est utilisée après la couche de convolution pour réduire la taille des données tout en préservant les informations importantes. Elle fonctionne en appliquant une fenêtre qui se déplace sur l'image, puis récupère soit la valeur maximale (max pooling), soit la moyenne des valeurs (average pooling). Cette réduction de données diminue le nombre de calculs et améliore l'invariance à la translation, en permettant de détecter un motif dans une zone plus large, plutôt qu'à une position exacte. Un padding peut être appliqué si la taille de la fenêtre ne correspond pas parfaitement à l'entrée. [8] [9] [11]

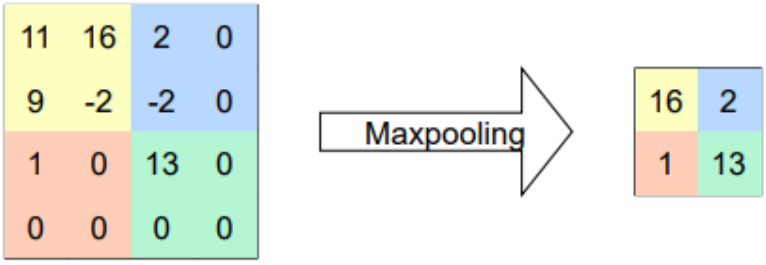


***Figure 2.15* :** Processus de Max-Pooling

2.4.4 Architecture globale d'un réseau convolutionnel

Les couches convolutionnelles peuvent être empilées pour former un réseau ; elles sont accompagnées de couches dites de pooling qui permet de réduire la taille des cartes de caractéristiques, et de couches de batch normalization qui effectuent un recentrage et une normalisation des données. Pour les tâches de classification, le réseau se termine par une ou plusieurs couches entièrement connectées (fully connected layers). [11]

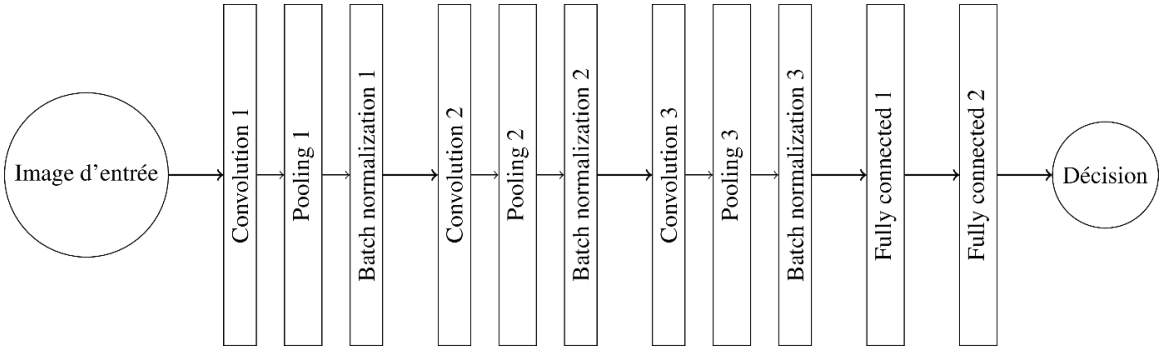


Figure 2.16 : Architecture d'un CNN

L'architecture typique d'un CNN comprend plusieurs couches principales :

- La couche de convolution (CONV) analyse l'image en entrée pour détecter des caractéristiques (features). Importantes Elle génère en sortie des cartes de caractéristiques (feature maps), qui représentent ces éléments détectés dans l'image.
- La couche de Pooling (POOL) est appliquée entre deux couches de convolution, elle réduit la taille des cartes de caractéristiques tout en conservant les informations clés. Elle utilise des techniques comme le max-pooling (valeur maximale) ou average pooling (valeur moyenne).
- La couche de batch normalization : aide à réduire le phénomène d'évanescence du gradient, surtout lorsque des fonctions comme ReLU ne sont pas utilisées. Elle maintient les valeurs d'un batch dans la zone non saturée d'une fonction d'activation, par exemple près de 0 pour la tangente hyperbolique. Cela améliore la stabilité et la vitesse d'apprentissage du modèle. [20]
- Les couches Fully Connected (FC) sont positionnées à la fin de l'architecture du CNN et sont connectées à tous les neurones de sortie, d'où leur appellation (d'où le terme fully-connected). Elles combinent les caractéristiques extraites et appliquent une fonction d'activation pour classifier l'image. Elles renvoient un vecteur de

probabilités, chaque valeur représentant la probabilité que l'image appartienne à une classe spécifique.

D'où, la convolution extrait les caractéristiques importantes de l'image (comme les contours et les formes) en appliquant des filtres. Ensuite, la couche de Pooling réduit la taille des cartes de caractéristiques, diminuant ainsi la complexité du modèle tout en conservant l'information essentielle. La batch normalization stabilise et accélère l'apprentissage en normalisant les données. Enfin, les couches fully connected utilisent ces caractéristiques pour classifier les images en leur attribuant des étiquettes.

2.5 Méthodologie et Processus de Développement

2.5.1 Processus de Reconnaissance des Gestes

Dans le cadre de la reconnaissance et de la classification d'images, il existe quatre tâches principales : la classification, la localisation, la détection, et la segmentation. Pour la reconnaissance des gestes, nous devons nous concentrer sur la classification, la localisation, et la détection. Voici les étapes que nous avons suivies :

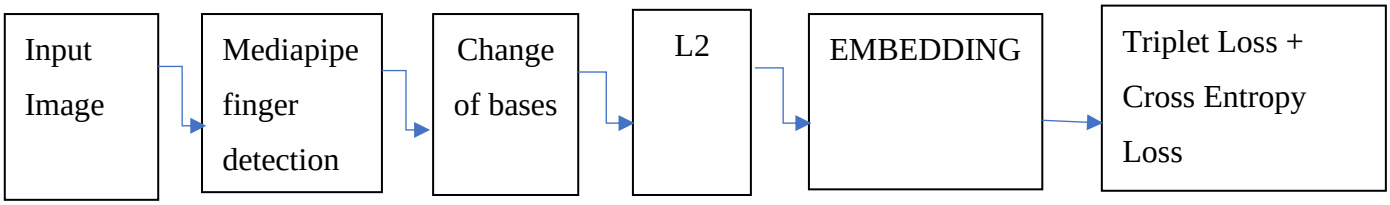


***Figure 2.17* :** Processus de mise en œuvre de la classification des gestes

- Classification et Localisation : Nous avons utilisé MediaPipe, un outil open-source de Google, pour obtenir la position et les coordonnées des points de la main. Cela nous permet de détecter et de localiser la main dans l'image.
- Prétraitement des données : Des transformations géométriques sont appliquées aux coordonnées de la main obtenues à l'étape précédente pour normaliser les données. Cette normalisation garantit que la forme et la position de la main soient comparables et cohérentes d'une image à l'autre.
- Extraction de caractéristiques (Feature Embedding) : Nous avons utilisé un réseau de neurones appelé FingerNet pour extraire des caractéristiques représentatives à partir des coordonnées normalisées de la main. FingerNet est conçu pour capturer les informations essentielles des points de la main.
- Fonctions de perte (Loss Functions) : Pendant l'entraînement du réseau, nous avons appliqué les fonctions de perte Triple Loss et CrossEntropy Loss. La Triple Loss permet

de rapprocher les représentations (embeddings) des gestes similaires dans l'espace d'embedding, tout en éloignant celles des gestes différents. La fonction CrossEntropy Loss, quant à elle, est utilisée pour la classification des gestes.[9]

2.5.2 Préparation des Données

Dans cette étude, la reconnaissance des gestes est basée sur les points clés des doigts. Ainsi, si le degré de flexion des articulations des doigts est le même, on peut considérer qu'il s'agit du même geste. Pour prendre en compte cette caractéristique, un jeu de données de gestes enregistrables ou Registerable Gesture Datase (RGDS) en anglais a été construit spécifiquement pour la reconnaissance des gestes.

Le jeu de données RGDS comprend des images de gestes de la main avec une résolution de 2624 × 2457 pixels. Il contient 32 catégories différentes de gestes, chacune avec 50 échantillons, pour un total de 1600 échantillons.[9]

2.5.3 Pretraitement des Donnees :

Après avoir traité les images des mains à travers le réseau MediaPipe, nous avons obtenu les coordonnées tridimensionnelles de 21 points clés des doigts, comme illustré dans la Figure 3. À ce stade :

- Les coordonnées x et y des points sont définies par rapport au coin inférieur gauche de l'image comme origine du repère.
- La coordonnée $\boldsymbol{z}$ est définie en prenant le point 0 (le poignet) comme origine.

Les transformations canoniques sont des transformations mathématiques souvent utilisées pour analyser et simplifier la description des systèmes physiques. Elles consistent à choisir un système de coordonnées approprié pour convertir les grandeurs physiques du système original, ce qui rend la formulation du problème plus concise ou plus facile à résoudre.

L'idée principale de ces transformations est de sélectionner un nouveau jeu de vecteurs de base pour représenter les vecteurs dans le système de coordonnées original. Ces nouveaux vecteurs de base possèdent souvent des propriétés spéciales, comme l'orthogonalité ou la normalisation, ce qui permet de simplifier la description ou la résolution du problème.[9]

Dans ce cas précis, nous considérons deux ensembles de vecteurs de base :

1. Un basé sur les points de 0 à 5.
2. Un autre basé sur les points de 0 à 17.

En prenant le point 0 comme origine des coordonnées, nous avons effectué une translation des coordonnées originales. Soit x_0, y_0 les coordonnées originales du point 0, et x_i et y_i les coordonnées des points de 0 à 5 et de 0 à 17.

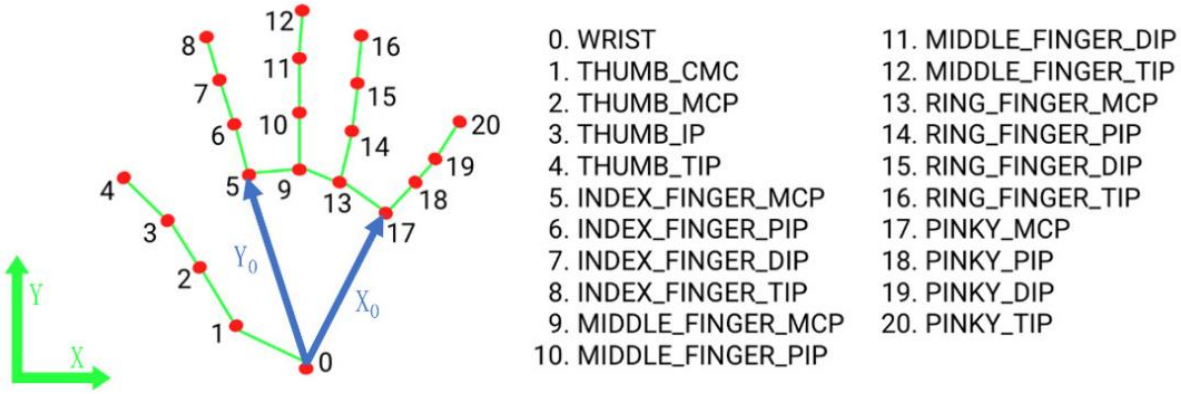


***Figure 2.18* :** Points de repère des doigts

2.5.4 Model de Construction

Après le prétraitement des données, les images de gestes sont transformées en des données de dimensions 1 × 22 × 3. Ces 22 dimensions contiennent les informations suivantes :

- La première dimension représente l'orientation de la main.
- Les dimensions 2 à 6 contiennent les coordonnées de chaque articulation du pouce.
- Les dimensions 7 à 10 contiennent les coordonnées de chaque articulation de l'index.
- Les dimensions 11 à 14 contiennent les coordonnées de chaque articulation du majeur.
- Les dimensions 15 à 18 contiennent les coordonnées de chaque articulation de l'annulaire.
- Les dimensions 19 à 22 contiennent les coordonnées de chaque articulation du petit doigt.

Ces caractéristiques de gestes sont ensuite extraites et combinées à l'aide du bloc FingerComb pour former des données de dimension 1 × 33 × 32. La structure du bloc FingerComb est illustrée dans la Figure 4, où la convolution 1D (Conv1D) est utilisée pour traiter les données.

-Innovation de ResNet-16 :

La principale innovation de ResNet-16 réside dans l'introduction des connexions résiduelles. Ces connexions résiduelles résolvent les problèmes de disparition de gradient et de dégradation du modèle lors de l'entraînement des réseaux de neurones profonds. Cette connexion de raccourci ajoute directement les caractéristiques d'entrée aux caractéristiques de sortie, permettant ainsi à l'information de circuler plus facilement dans le réseau. Cela facilite l'entraînement et optimise le modèle de manière plus efficace.

Grâce à ces connexions, le modèle peut mieux apprendre les caractéristiques complexes des gestes tout en évitant les problèmes liés à la profondeur du réseau.[9]

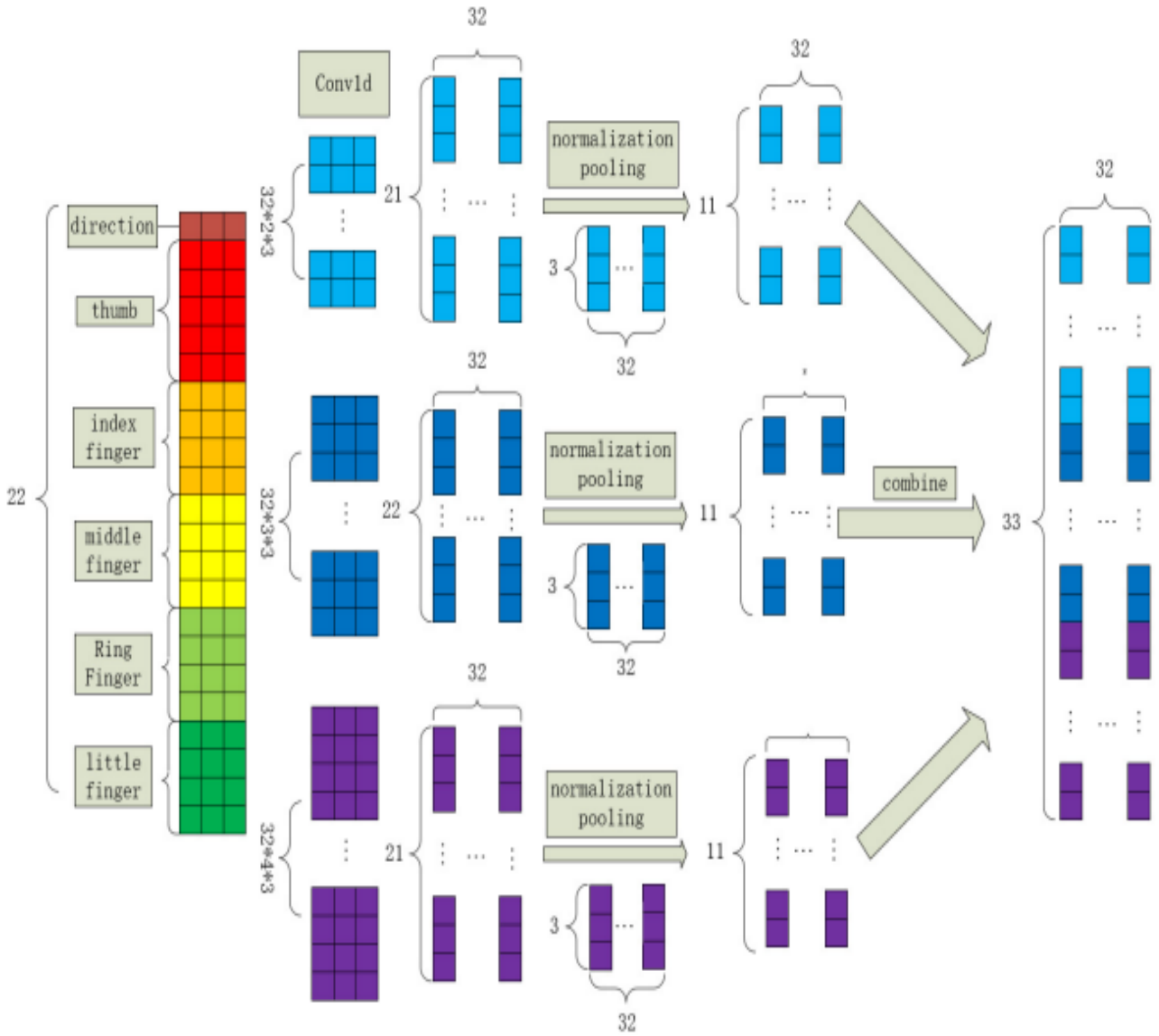


Figure 2.19 : Architecture CNN pour la détection et le suivi précis des mains

Les paramètres du modèle sont présentés dans le Tableau 1. FingerNet utilise des données de dimensions 1 × 22 × 3 en entrée et, grâce à des convolutions, traite les caractéristiques d'entrée pour produire des résultats en sortie.

Layer Name	Filter	Output Size
Input		$1 \times 22 \times 3$
FingerComb	$\begin{bmatrix} 1 \times 2, & 32 & s = 1, p = 0 \\ 1 \times 2, & 32 & s = 1, p = 0 \end{bmatrix}$ $\begin{bmatrix} 1 \times 3, & 32 & s = 1, p = 1 \\ 1 \times 3, & 32 & s = 1, p = 1 \end{bmatrix}$ $\begin{bmatrix} 1 \times 4, & 32 & s = 1, p = 1 \\ 1 \times 4, & 32 & s = 1, p = 1 \end{bmatrix}$	$1 \times 33 \times 32$
Conv2_x	$\begin{bmatrix} 1 \times 3, & 32 & s = 1, p = 1 \\ 1 \times 3, & 32 & s = 1, p = 1 \end{bmatrix} \times 2$	$1 \times 33 \times 32$
Conv3_x	$\begin{bmatrix} 1 \times 3, & 64 & s = 2, p = 1 \\ 1 \times 3, & 64 & s = 1, p = 1 \end{bmatrix}$ $\begin{bmatrix} 1 \times 3, & 64 & s = 1, p = 1 \\ 1 \times 3, & 64 & s = 1, p = 1 \end{bmatrix}$	$1 \times 17 \times 64$
Conv4_x	$\begin{bmatrix} 1 \times 3, & 128 & s = 2, p = 1 \\ 1 \times 3, & 128 & s = 1, p = 1 \end{bmatrix}$ $\begin{bmatrix} 1 \times 3, & 128 & s = 1, p = 1 \\ 1 \times 3, & 128 & s = 1, p = 1 \end{bmatrix}$	$1 \times 9 \times 128$
Conv5_x	$\begin{bmatrix} 1 \times 3, & 256 & s = 2, p = 1 \\ 1 \times 3, & 256 & s = 1, p = 1 \end{bmatrix}$ $\begin{bmatrix} 1 \times 3, & 256 & s = 1, p = 1 \\ 1 \times 3, & 256 & s = 1, p = 1 \end{bmatrix}$	$1 \times 5 \times 256$
Avgpool	256 K = 3, s = 1, p = 1	$1 \times 1 \times 256$
FC	(256, 128)	1×128
FC	(128, 32)	1×32

Tableau 2.01: Architecture Hiérarchique de FingerNet : Cadre Neuronal pour la Reconnaissance des Gestes de la Main"

Ce tableau décrit l'architecture d'un réseau neuronal conçu pour traiter des données de points de repère (landmarks) de la main. Voici une explication détaillée des paramètres :

- Input (Entrée) :
  - Taille : $1 * 22 * 3$
  - Cela représente probablement 22 points de repère sur la main, chacun avec 3 coordonnées (x, y, z).
- FingerComb :
  - Cette couche utilise différents filtres de convolution 1D pour traiter les données d'entrée.
  - Les filtres varient en taille $(1 * 2;\ 1 * 3;\ 1 * 4)$ avec 32 canaux chacun.
  - 's' représente le stride (pas), toujours égal à 1.
  - 'p' est le padding, variant entre 0 et 1.
  - La sortie est de taille $1 * 33 * 32$, augmentant la dimension des caractéristiques.
- Conv2_x à Conv5_x :
  - Ces couches sont des blocs de convolution empilés.

- Chaque bloc augmente progressivement le nombre de canaux ($32 \rightarrow 64 \rightarrow 128 \rightarrow 256$) tout en réduisant la dimension spatiale.
- Les filtres sont principalement de taille $1 * 3$.
- Le stride alterne entre 1 et 2, permettant la réduction de la dimension spatiale.
- Le padding est maintenu à 1 pour préserver l'information aux bords.

➢ Avgpool (Average Pooling) :

- Réduit la dimension spatiale à $1 * 1$, gardant 256 canaux.

➢ FC (Fully Connected) :

- Deux couches entièrement connectées réduisent progressivement la dimension des caractéristiques.
- La première passe de 256 à 128 neurones.
- La seconde réduit de 128 à 32 neurones, qui est probablement la dimension finale de l'embedding ou le nombre de classes de gestes.

Cette architecture est conçue pour extraire efficacement des caractéristiques hiérarchiques à partir des points de repère de la main, en commençant par des caractéristiques locales (FingerComb) et en progressant vers des représentations plus globales et abstraites (couches Conv et FC). Le résultat final est une représentation compacte de 32 dimensions, idéale pour la classification ou la reconnaissance de gestes.

Conclusion

En conclusion, ce chapitre a permis de mieux comprendre l'application des réseaux de neurones dans la reconnaissance des gestes, en mettant l'accent sur leur fonctionnement et sur l'importance des réseaux convolutifs (CNN) dans le traitement des images et la classification des gestes.

CHAPITRE 3
REALISATION DU SYSTEME HOMME-MACHINE CONROLE PAR DES GESTES

3.1 Introduction

Après avoir défini des concepts clés comme la vision artificielle et l'apprentissage profond dans les chapitres précédents, nous pouvons maintenant les appliquer pour créer un système de contrôle par gestes de la main. Grâce aux avancées de la vision par ordinateur, aux caméras haute résolution, et aux algorithmes de suivi de mouvements, il est possible de développer des interfaces sans contact, contrôlées par les gestes. Cela rend l'interaction plus intuitive et naturelle pour l'utilisateur.

Ce chapitre traite de la conception et de la mise en œuvre d'un système qui permet de contrôler la souris à partir de gestes. Nous expliquerons comment le système fonctionne, quels gestes sont utilisés pour chaque action, et quels outils technologiques ont été employés.

3.2 Description de projet

Notre projet représente une avancée significative dans l'interaction homme-machine, développant une interface gestuelle sophistiquée basée sur la vision par ordinateur. Au cœur du système, un algorithme de reconnaissance de gestes en temps réel exploite des techniques avancées de traitement d'image et d'apprentissage automatique pour transformer les mouvements de la main en commandes précises de l'interface utilisateur.

L'architecture intègre un module de détection de main haute performance utilisant des réseaux de neurones convolutifs, un système de tracking multi-points pour une précision accrue, et un moteur d'interprétation de gestes capable de simuler diverses actions (déplacement du curseur, clics, défilement). L'accent est mis sur la minimisation de la latence et l'optimisation des performances pour une expérience utilisateur fluide.

Ce système ouvre des perspectives révolutionnaires dans plusieurs domaines : en domotique pour une interaction sans contact avec les appareils connectés, dans le contexte médical pour une interaction aseptique, et en accessibilité pour permettre aux personnes à mobilité réduite de contrôler des systèmes informatiques avec aisance. Dans l'industrie du jeu vidéo et de la réalité virtuelle, notre technologie promet une immersion plus profonde et naturelle.

3.3 Architecture Globale du système :

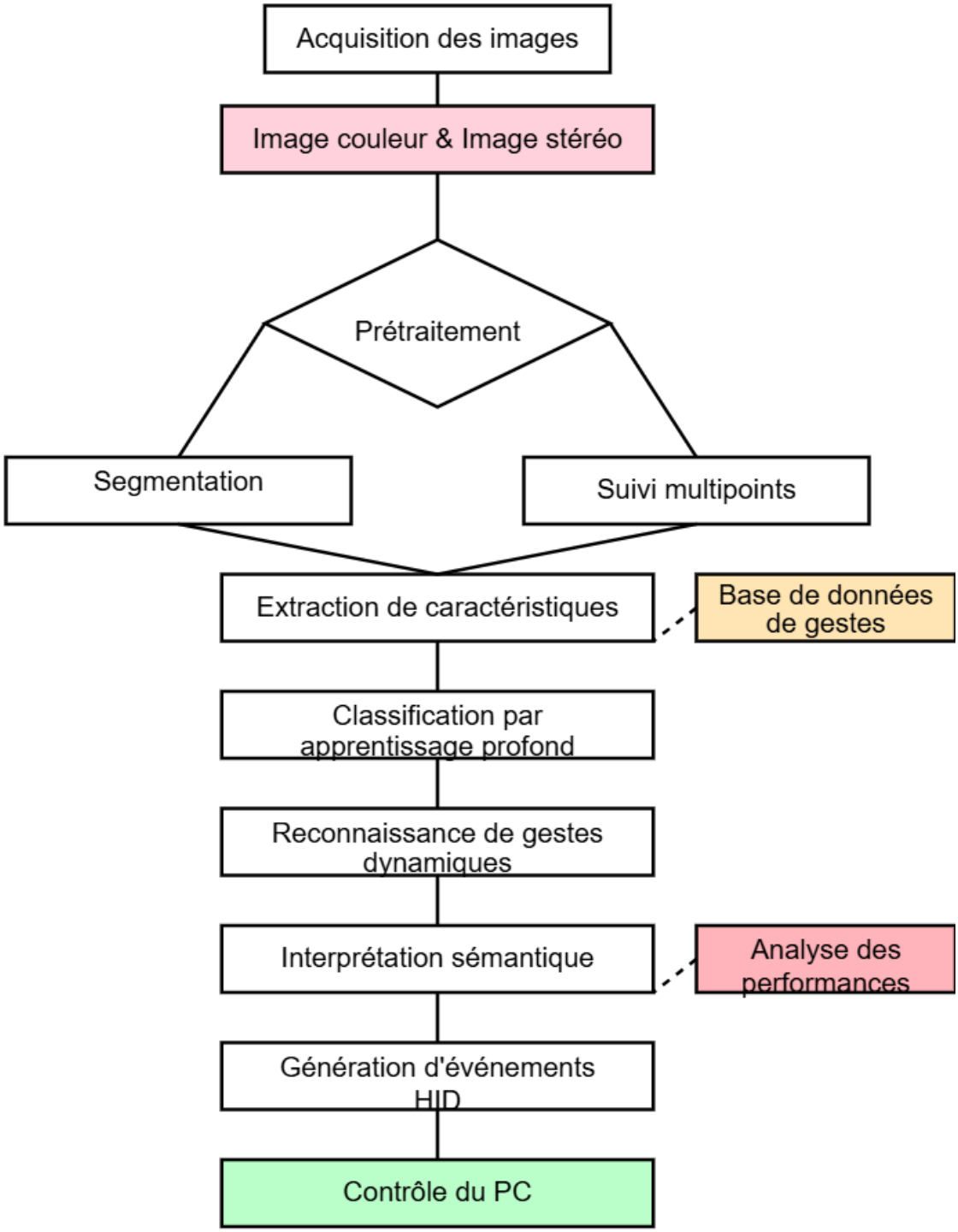


Figure 3.01 **:** Architecture Globale de notre système

3.3.1 L'acquisition des images

Comme nous l'avons vu dans le premier chapitre sur le système de traitement d'image (section 1.4.5), l'acquisition d'une image consiste à capturer une image à l'aide d'un dispositif tel qu'une caméra stéréo ou une webcam, puis à la convertir en données numériques pour qu'elle puisse être traitée par un ordinateur. Le prétraitement permet de corriger les dégradations survenues lors de l'acquisition.

Dans le cas de notre application contrôlée par gestes de la main, la première étape consiste en un processus d'identification qui débute par la capture d'une image à l'aide de la webcam. Cette image est ensuite traitée pour détecter les gestes de la main en analysant leurs caractéristiques spécifiques, afin de les reconnaître correctement.

3.3.2 La segmentation

La deuxième étape c'est la méthode de segmentation par division et fusion (ou *split and merge* en anglais) fait partie des techniques qui manipulent directement les régions d'une image. Le processus commence généralement par une première partition de l'image, qui est ensuite ajustée en divisant ou en fusionnant les régions. Une autre approche consiste à partir de quelques régions initiales, qui s'étendent progressivement par l'ajout de pixels jusqu'à ce que l'ensemble de l'image soit couvert. Dans ce cas, on parle de segmentation par croissance de régions. La figure suivante illustre le fonctionnement de cette méthode.

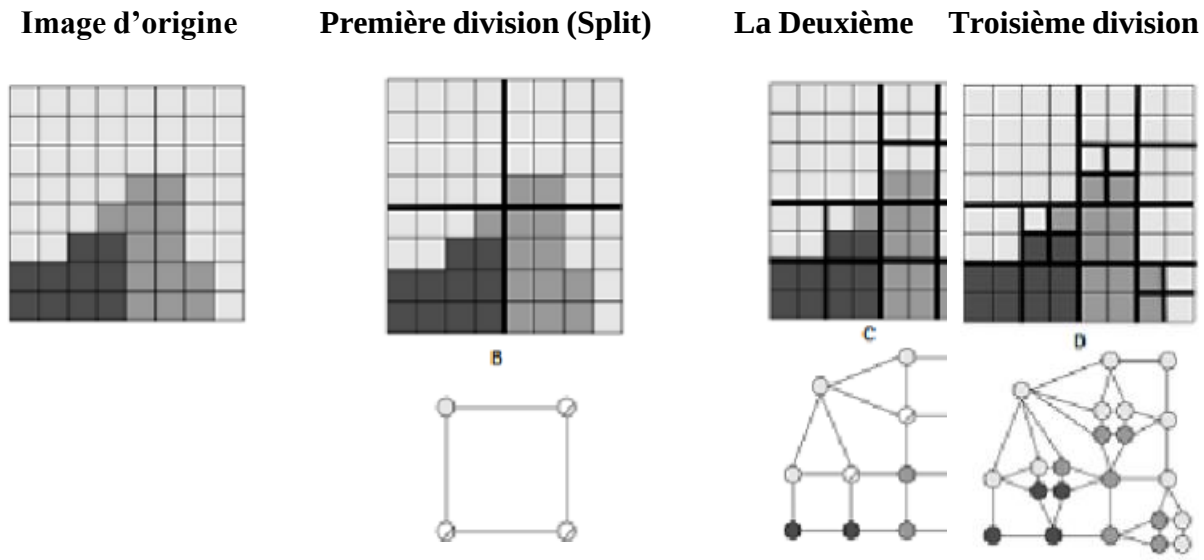


***Figure 3.02* :** Le processus de fonctionnement de segmentation et leurs résultats

3.3.3 Extraction des caractéristiques (troisième étapes)

Après avoir détecté la paume dans l'image, la troisième étape consiste à extraire les caractéristiques. Ce processus génère 21 points clés pour chaque main détectée. Ces points clés, illustrés dans la figure ci-dessous, représentent les articulations de la main en 3D. Ils sont prédits directement comme des coordonnées 3D, ce qui permet de modéliser la structure de la main, même si celle-ci est partiellement cachée ou si certaines parties se chevauchent.

3.3.4 Classification et reconnaissance des gestes :

Le modèle de classification et de reconnaissance des gestes basé sur des réseaux de neurones convolutifs (CNN) est souvent utilisé dans les systèmes de recommandation et pour la détection d'objets. Ce type de modèle divise une image en plusieurs petites parties (ou "grilles"), et chaque partie est chargée de détecter un objet qui pourrait s'y trouver. L'avantage de ce modèle est qu'il peut détecter des objets en temps réel avec peu de ressources informatiques.

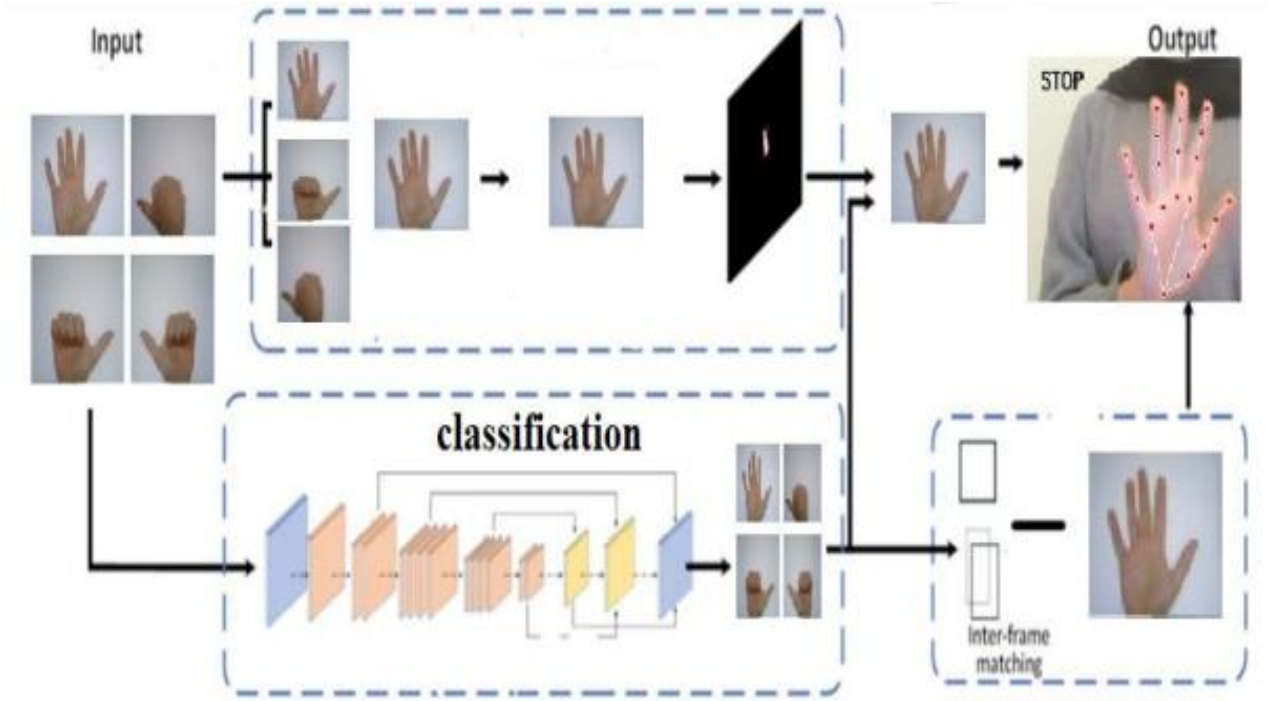


***Figure 3.03* :** Architecture global de classification et reconnaissance des gestes

Récapitulations

L'architecture globale de ce système de classification et reconnaissance des gestes fonctionne en plusieurs étapes :

- Entrée d'images : Le système reçoit plusieurs images de mains dans différentes positions.

- Prétraitement : Ces images sont traitées pour ne garder que les informations utiles (comme l'image de la main isolée).
- Classification : Un réseau de neurones convolutif (CNN) analyse ces images pour identifier les gestes.
- Détection des points clés : Le système identifie des points importants sur la main (comme le bout des doigts).
- Suivi temporel : Les mouvements de la main sont suivis d'une image à l'autre pour comprendre le geste dans son ensemble.
- Résultat : Le système produit une image finale montrant la main avec les points clés identifiés et le geste reconnu.

3.4 Outils utilisés et environnement de développement

3.4.1 Python

Python est un langage de programmation créé par Guido van Rossum en 1991. C'est un langage très populaire car il est simple à lire, facile à apprendre et utilisé dans de nombreux domaines comme le développement web, la science des données, l'intelligence artificielle, l'automatisation, et bien d'autres. Python est un langage interprété, ce qui signifie que le code peut être exécuté directement sans compilation préalable. Ce qui rend Python particulièrement puissant, c'est sa syntaxe claire et sa vaste bibliothèque standard, qui permet aux développeurs de gagner du temps en utilisant des outils prêts à l'emploi pour diverses tâches.

Dans notre projet de contrôle de souris par gestes, nous utilisons OpenCV pour capturer des images en temps réel, Mediapipe pour détecter les mouvements de la main, PyAutoGUI pour simuler les actions de la souris, et Tkinter pour créer une interface utilisateur. L'architecture générale inclut la capture d'images, la détection des gestes, puis leur classification pour déclencher la souris.

***Figure 3.04* :** Logo du langage python

3.4.2 OpenCv

OpenCV (Open Source Computer Vision Library) est une bibliothèque open source largement utilisée pour le traitement d'images et la vision par ordinateur. Elle permet de capturer et

manipuler des images et des vidéos en temps réel, comme dans notre projet où elle est utilisée pour capturer le flux vidéo de la webcam et afficher les résultats visuels à l'utilisateur. Elle permet aussi de dessiner des repères (landmarks) sur la main, facilitant ainsi la détection des doigts et l'interaction avec l'écran.

Elle propose un large éventail d'outils pour détecter et suivre des objets, reconnaître des formes, calibrer des caméras, et même analyser des images. Bien qu'elle soit écrite en C++, elle dispose de nombreuses interfaces pour des langages comme Python et Java, ce qui la rend très flexible. Elle est utilisée dans divers domaines, comme la robotique, la réalité augmentée, l'analyse d'images médicales ou la surveillance. Son efficacité et ses algorithmes optimisés en font un choix populaire parmi les chercheurs et développeurs qui travaillent sur des projets de vision par ordinateur.

***Figure 3.05* :** Logo de la bibliothèque OpenCv

3.4.3 Tkinter

Tkinter est une bibliothèque standard de Python qui permet de créer des interfaces graphiques (GUI) de manière simple et efficace. Très facile à utiliser, elle est souvent employée pour concevoir des interfaces interactives en Python.

Dans notre projet de contrôle de la souris par gestes, Tkinter joue un rôle essentiel en fournissant une interface graphique pratique et intuitive. Elle permet à l'utilisateur de gérer le système sans passer par des lignes de commande compliquées en fournissant une interface graphique intuitive pour simplifier l'utilisation du système de contrôle par gestes.

***Figure 3.06* :** Logo de la bibliothèque Tkinter

3.4.5 Numpy

NumPy NumPy est une bibliothèque Python qui facilite la manipulation de tableaux multidimensionnels, appelés matrices. Elle propose de nombreuses fonctions mathématiques pour effectuer des opérations sur ces tableaux. Open source et gratuite, NumPy permet de créer des tableaux à partir de fichiers, de les enregistrer, et de manipuler des vecteurs, matrices et polynômes. C'est également une base essentielle pour SciPy, qui regroupe des bibliothèques dédiées au calcul scientifique.

Dans notre projet de contrôle de la souris par gestes, NumPy est utilisé pour traiter et analyser efficacement les données issues des mouvements de la main. Grâce à ses capacités de manipulation de tableaux, nous pouvons stocker les coordonnées des points clés détectés par la caméra et effectuer des calculs mathématiques nécessaires à la reconnaissance des gestes. Cela permet d'optimiser les performances du système et de garantir une détection précise des mouvements.

.

***Figure 3.07* :** Logo de la bibliothèque Numpy

3.4.6 MediaPipe

MediaPip**e** est un framework open source de Google qui facilite le développement d'applications de vision par ordinateur. Il propose des outils prêts à l'emploi pour des tâches comme le suivi des mains, la détection d'objets, la reconnaissance faciale, et bien plus encore. Conçu pour traiter des flux vidéo en temps réel, MediaPipe est flexible et peut être utilisé sur différentes plateformes, des téléphones aux ordinateurs. Sa modularité permet de créer des pipelines personnalisés en combinant facilement différents modules. MediaPipe est largement utilisé dans la recherche et l'industrie pour des projets d'intelligence artificielle.

Dans notre projet, MediaPipe détecte et suit la position des mains et des doigts en temps réel, permettant à l'utilisateur de contrôler la souris par des gestes.

***Figure 3.08* :** Logo de la bibliothèque Mediapip

3.4.7 Pyatougui

PyAutoGUI est une bibliothèque Python utilisée pour automatiser les actions de la souris et du clavier sur un ordinateur. Elle permet de simuler des clics, des mouvements de la souris, des saisies au clavier et d'autres interactions utilisateur, rendant possible le contrôle automatique de l'interface graphique d'un système. PyAutoGUI est compatible avec plusieurs systèmes d'exploitation, notamment Windows, macOS et Linux. Cette bibliothèque est particulièrement utile pour automatiser des tâches répétitives, créer des scripts de test pour les interfaces utilisateur, ou, comme dans notre projet, simuler des actions en réponse à des gestes détectés.

Dans notre système de contrôle de la souris par gestes, PyAutoGUI est utilisé pour simuler les actions de la souris en fonction des mouvements de la main capturés par la webcam. Par exemple, lorsqu'un utilisateur bouge sa main, PyAutoGUI déplace le curseur selon la position des doigts. Il permet aussi de déclencher un clic gauche ou droit en fonction des gestes (lever l'index pour un clic gauche, lever l'auriculaire et l'index pour un clic droit).

3.4 Etapes et processus du projet

La reconnaissance des gestes est un domaine en plein essor dans l'interaction homme-machine, avec des applications variées comme le contrôle d'environnements virtuels, la traduction en langue des signes, le pilotage de robots ou la création musicale. Dans ce projet, nous nous concentrons spécifiquement sur le contrôle d'un PC à l'aide de gestes de la main.

Notre objectif est de développer un système en temps réel capable de reconnaître des gestes de la main en utilisant MediaPipe comme framework principal, et en s'appuyant sur OpenCV, PyAutoGUI et Tkinter pour l'interface utilisateur et les interactions avec le PC.

Les principales étapes du projet sont les suivantes :

- Importer les bibliothèques nécessaires.
- Initialiser les modèles de reconnaissance des gestes.
- Capturer les images via une webcam.
- Détecter les points clés de la main.

- Reconnaître les gestes et les associer aux actions de la souris, comme le clic ou le défilement.

3.4.1 Importer les packages nécessaires

Pour créer ce système de contrôle de la souris par des gestes, nous avons besoin d'importer plusieurs bibliothèques essentielles. D'abord, OpenCV (cv2) est utilisé pour capturer les images depuis la webcam et afficher les résultats. Ensuite, NumPy permet de gérer les opérations mathématiques nécessaires au traitement des coordonnées des points de repère. Nous utilisons également Mediapipe, un framework qui nous aide à détecter et suivre les mains en temps réel. Enfin, PyAutoGUI nous permet de simuler les actions de la souris (déplacement, clic, scroll) en fonction des gestes reconnus, et Tkinter pour l'interface graphique.

3.4.2 Initialiser les modèles

Le modèle de reconnaissance des mains est initialisé à l'aide du module MediaPipe Hands. Ce module est spécialement conçu pour la détection des mains et de leurs points clés (landmarks). Nous créons un objet mpHands qui contient la configuration du modèle. En utilisant la méthode mpHands.Hands(), nous spécifions les paramètres, comme le nombre maximum de mains à détecter dans une seule image (max_num_hands=1 dans notre cas). Le module mp.solutions.drawing_utils est utilisé pour dessiner automatiquement les points de repère détectés sur la main, évitant ainsi de les tracer manuellement.

3.4.3 Lire les images d'une webcam

Pour capturer les images en temps réel, nous créons un objet VideoCapture via OpenCV en utilisant l'argument 0, qui correspond à la première webcam détectée sur le système. Si plusieurs caméras sont disponibles, cet argument peut être ajusté en fonction de l'ID de la caméra. La méthode cap.read() lit chaque image capturée par la webcam, puis la méthode cv2.flip() retourne horizontalement l'image pour imiter une vue miroir (ce qui est plus intuitif pour le contrôle par gestes). Enfin, la méthode cv2.imshow() affiche l'image dans une fenêtre OpenCV, et cv2.waitKey() maintient cette fenêtre ouverte jusqu'à ce qu'une touche soit pressée (par exemple 'q' pour quitter).

3.4.4 Détecter les points clés de la main

Puisque MediaPipe attend des images au format RVB tandis qu'OpenCV capture en BGR, nous utilisons la fonction cv2.cvtColor() pour convertir les images capturées au bon format. Ensuite, la fonction process() de MediaPipe traite cette image et renvoie les résultats, incluant les points de repère (landmarks) des mains. Nous vérifions si une main a été détectée à l'aide de

result.multi_hand_landmarks. Chaque point de repère est représenté par des coordonnées normalisées entre 0 et 1, que nous transformons en pixels en multipliant ces valeurs par les dimensions de l'image. Les points de repère sont ensuite tracés sur l'image à l'aide de la fonction mpDraw.draw_landmarks().

3.4.5 Reconnaître les gestes de la main

Le système utilise MediaPipe pour reconnaître les gestes de la main capturés via une caméra (comme Kinect). Le processus commence par l'acquisition d'une image de la main. Ensuite, MediaPipe détecte la paume grâce à un Palm Detector, puis identifie les points clés (landmarks) de la main, comme les articulations des doigts. Ces points sont utilisés pour suivre la main en 3D et déterminer les gestes effectués.

Chaque combinaison de doigts levés ou abaissés est reconnue comme un geste particulier. Par exemple :

- Un clic gauche est associé au majeur levé.
- Un clic droit est déclenché quand l'index et l'auriculaire sont levés.
- Le déplacement du curseur est contrôlé par les mouvements de l'index levé.
- Le défilement (scroll) est géré par le pouce et l'index levés, et leur distance permet de déterminer la direction du scroll.
- La capture d'écran est déclenchée en fermant le poing pendant 4 secondes.

Une fois un geste reconnu, une action associée est exécutée à l'aide de PyAutoGUI, comme simuler un clic ou déplacer le curseur. Dans certaines applications, un menu s'ouvre en fonction du geste détecté, permettant de naviguer et interagir sans clavier ni souris.

Le schéma de la figure 3.04 illustre ce processus étape par étape, de la détection de la main à l'exécution des actions basées sur les gestes

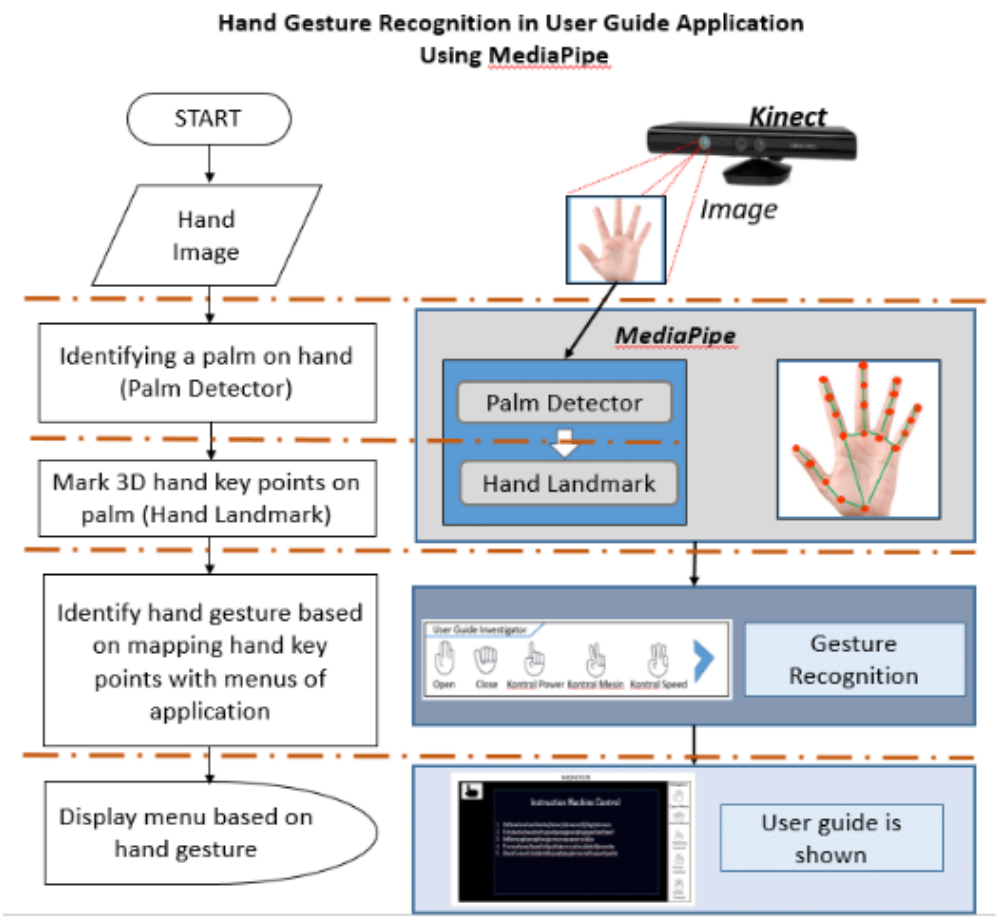


Figure 3.09 **:** Architecture de Reconnaissance des Gestes Basée sur MediaPipe

Conclusion

Pour conclure, ce chapitre a montré la faisabilité de la mise en œuvre d'un système de contrôle de la souris par des gestes de la main, tout en détaillant les différentes étapes techniques nécessaires à sa réalisation. Grâce à l'utilisation de technologies comme OpenCV et MediaPipe, ce système offre une interaction homme-machine intuitive et sans contact. Toutefois, certaines limitations, comme la précision du suivi en temps réel, méritent d'être optimisées pour améliorer l'efficacité et les performances globales du système.

CHAPITRE 4 : EVALUATION ET INTERPRETATION DES RESULTATS DU SYSTEME DE RECONNAISSANCE GESTUELLE

4.1 Représentation de l'application

4.1.1 Interface de l'application

Dans cette section, nous allons présenter les différentes interfaces que les utilisateurs peuvent utiliser pour faciliter l'interaction entre l'homme et la machine, ainsi que pour configurer le système. Il y a deux types d'interfaces principales que nous allons détailler :

4.1.1.1 Interface de configuration du système

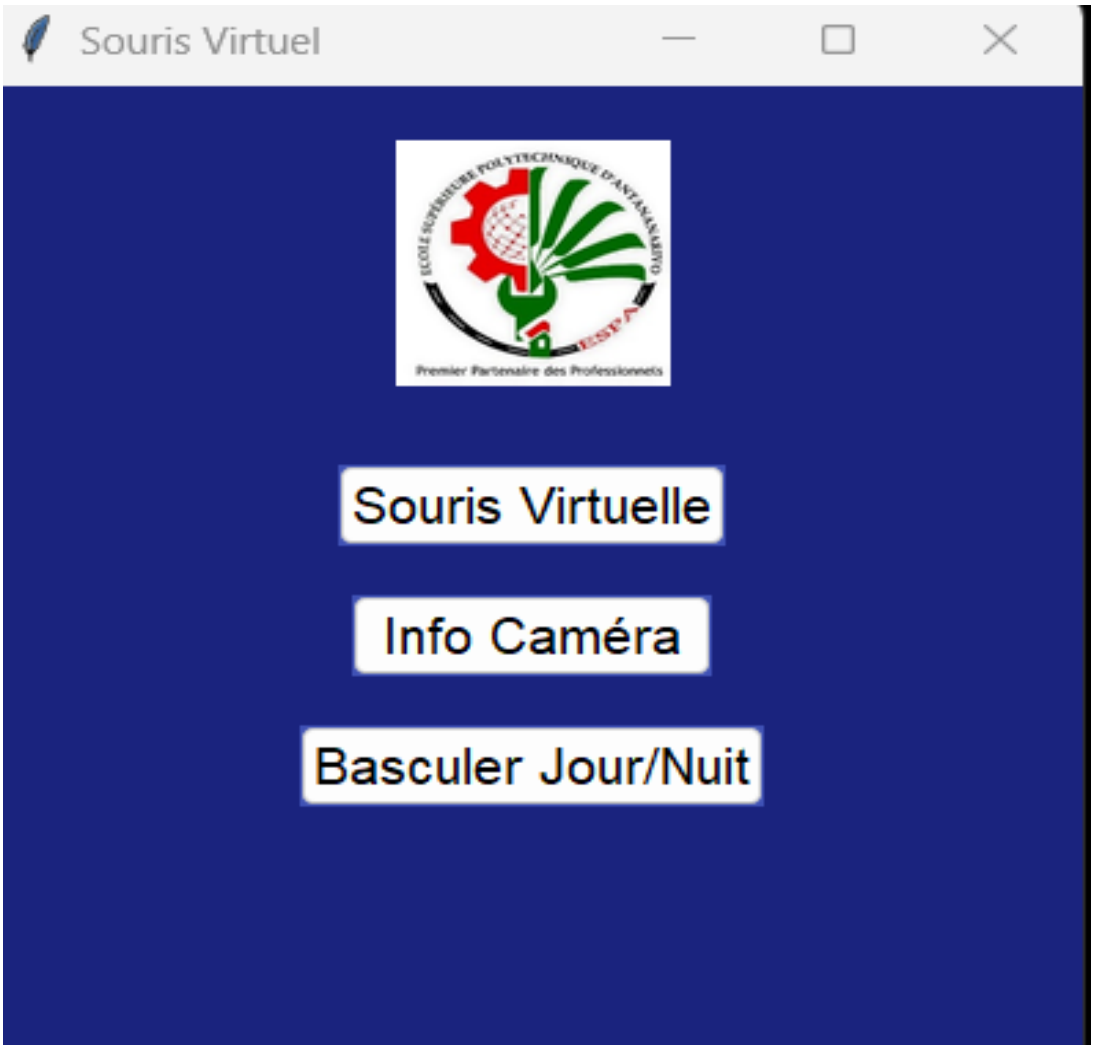


***Figure 4.01* :** Interface de la souris virtuelle

Cette interface permet de régler les paramètres de la caméra et de la souris virtuelle avant de lancer l'application. Elle comporte trois sections principales :

- Informations sur la caméra

Dans cette section, l'utilisateur peut configurer la caméra utilisée pour détecter les gestes. Par défaut, la webcam du PC est utilisée et identifiée par l'index "0". Si l'utilisateur dispose de caméras externes supplémentaires, il peut choisir leur numéro (1, 2, etc.). Il est également possible de régler la résolution de la caméra : par exemple, une webcam par défaut a souvent une largeur de 640 pixels et une hauteur de 480 pixels, adaptée à l'écran selon l'angle de la caméra.

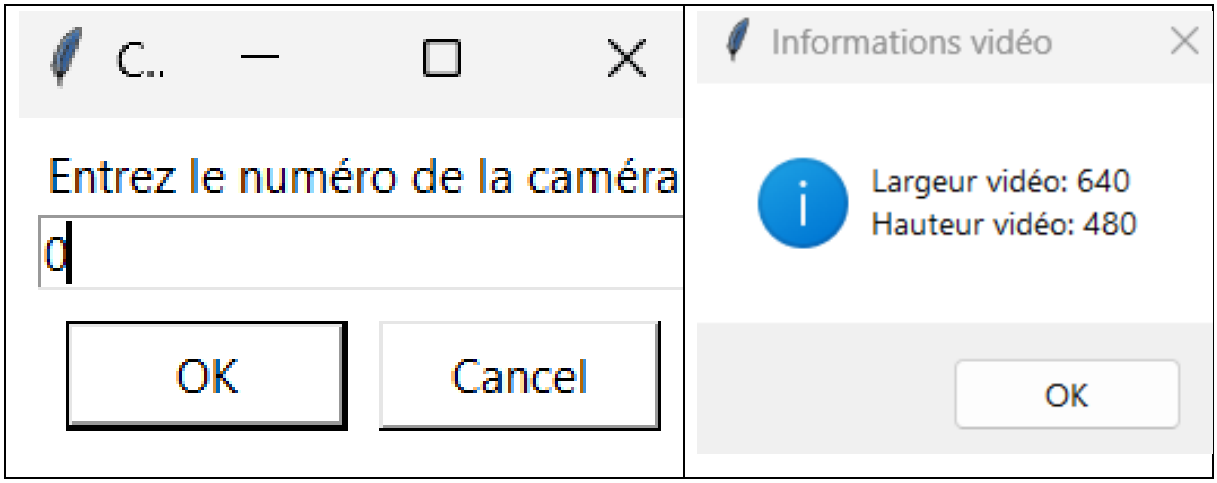


Figure 4.02 **:** Paramètres de la caméra active

- Souris virtuelle|

Avant de démarrer l'application, l'utilisateur doit entrer les paramètres de la caméra : le numéro de la caméra, la largeur et la hauteur de l'image. Ensuite, il peut choisir d'afficher ou non le FPS (images par seconde). Le FPS est un indicateur du nombre d'images affichées par seconde sur l'écran, visible en haut à gauche de la fenêtre pendant l'exécution.

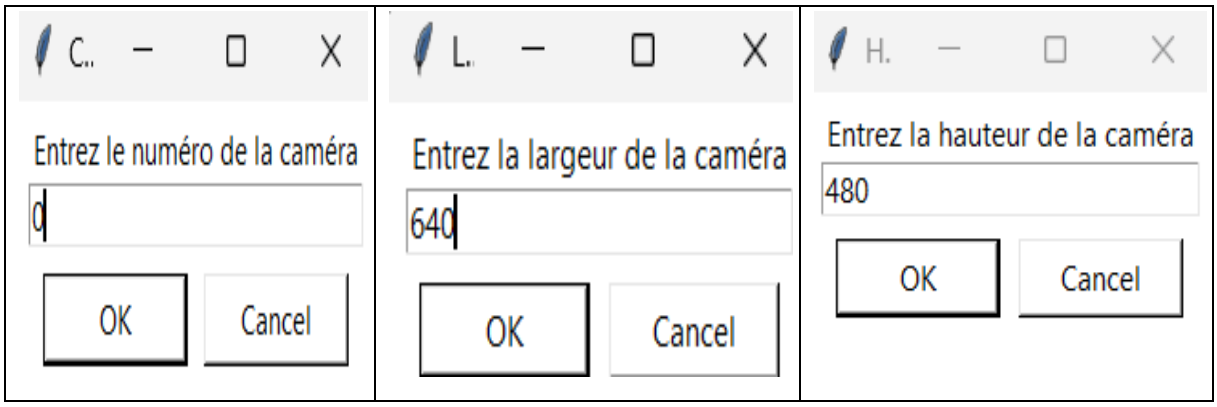


Figure 4.03 **:** Configuration de l'interface utilisateur du système

- Mode nuit ou jour

L'interface propose également un bouton pour basculer entre le mode nuit et le mode jour, afin d'adapter l'affichage aux préférences de l'utilisateur ou aux conditions d'éclairage.

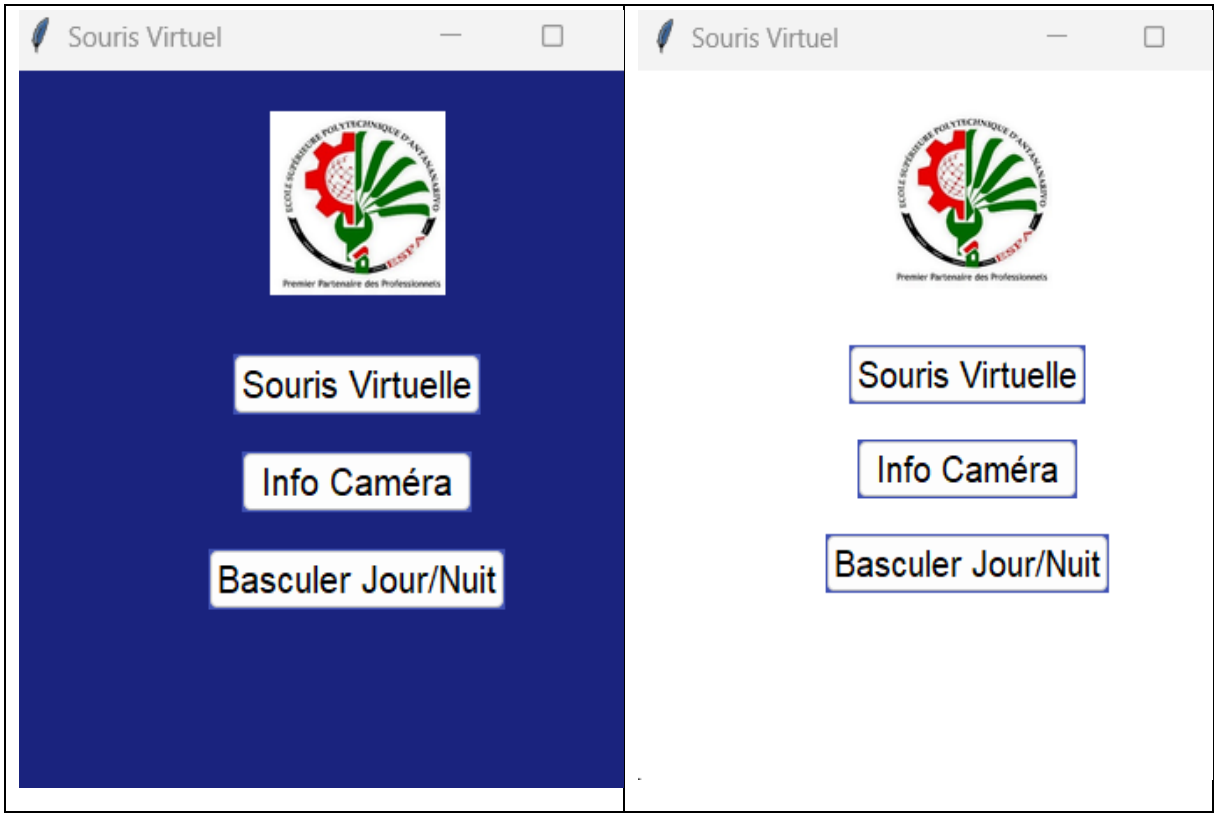


Figure 4.04 **:** Interface adaptative : modes diurnes et nocturne

4.1.1.2 Interface reconnaissance de gestes

Cette interface est le cœur de l'application. Elle permet de prédire les gestes de la main en temps réel et d'exécuter des actions en fonction de chaque geste détecté (comme un clic gauche, un clic droit ou un défilement). La prédiction du geste s'affiche en haut à droite de l'écran, tandis que le FPS est affiché en haut à gauche pour suivre la performance en temps réel, comme le montre la figure 3.16.

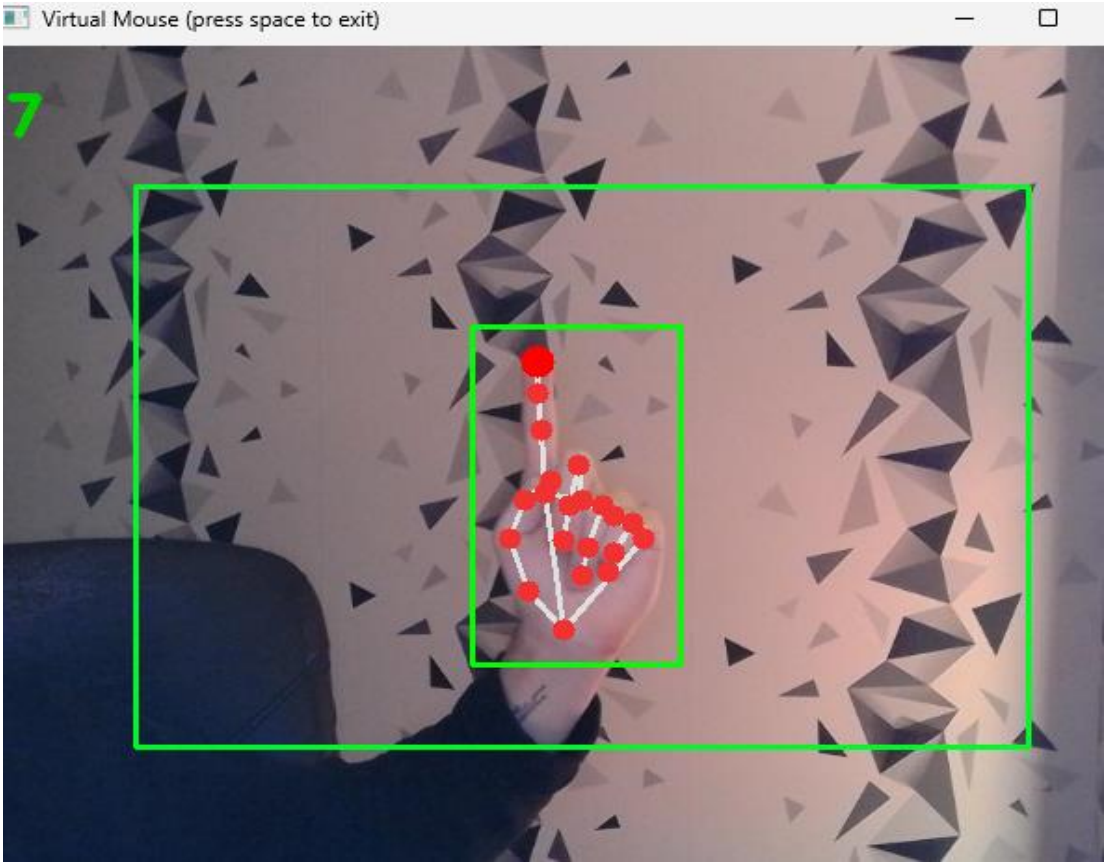


***Figure 4.05* :** Fenêtre de l'application Reconnaissance de geste

Pour faciliter l'utilisation du système, nous avons également ajouté une fonctionnalité simple pour arrêter le programme : il suffit d'appuyer sur la touche "Espace" pour fermer l'application.

4.1.2 But de l'application

L'objectif principal de cette application est de concevoir et mettre en œuvre un système innovant de souris virtuelle basé sur la vision par ordinateur. Ce projet vise à transformer les gestes de la main captés par une caméra en actions précises de contrôle du curseur, capture d'écran, offrant ainsi une alternative intuitive et sans contact aux périphériques d'entrée traditionnels. En exploitant des algorithmes avancés de détection et de suivi en temps réel, notre système cherche à offrir une expérience utilisateur fluide et réactive, tout en s'adaptant à divers environnements et conditions d'utilisation. Au-delà de la simple émulation d'une souris, ce projet explore les possibilités d'interaction homme-machine naturelle, ouvrant la voie à des applications potentielles dans des domaines variés tels que l'accessibilité, les environnements stériles, ou les interfaces immersives.

4.2 Présentation des tests

On affiche dans un tableau les résultats des tests réalisés avec l'application.

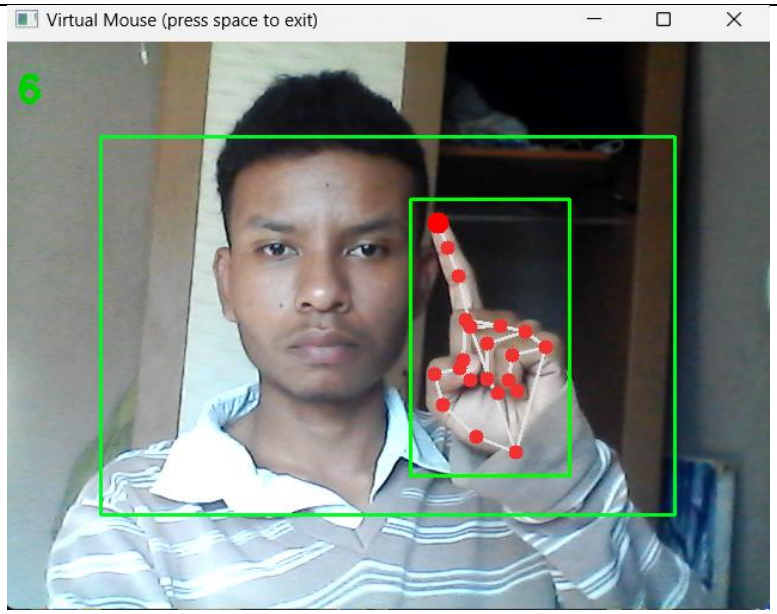
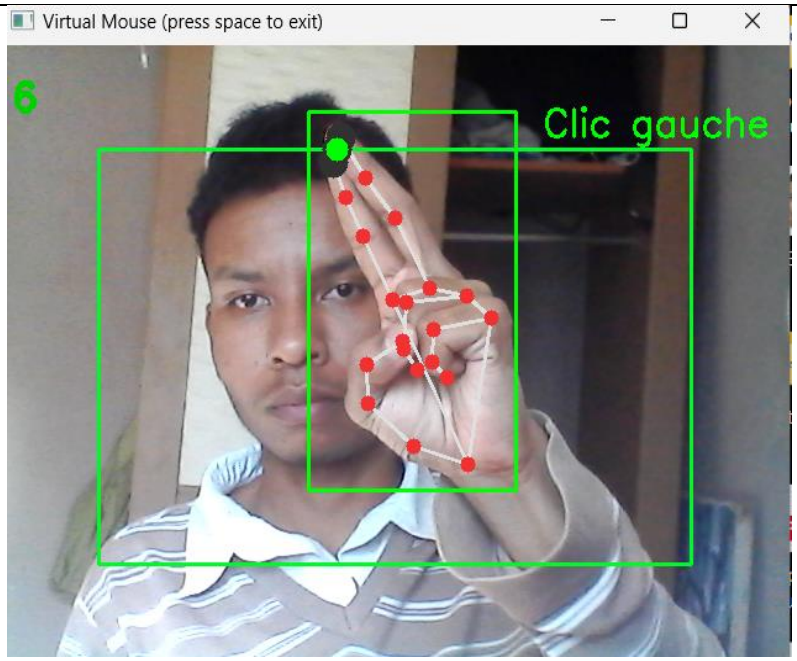
Action	Résultat
« Déplacement Curseur »	
« Click gauche »	

Tableau 2.02: Résultats des tests

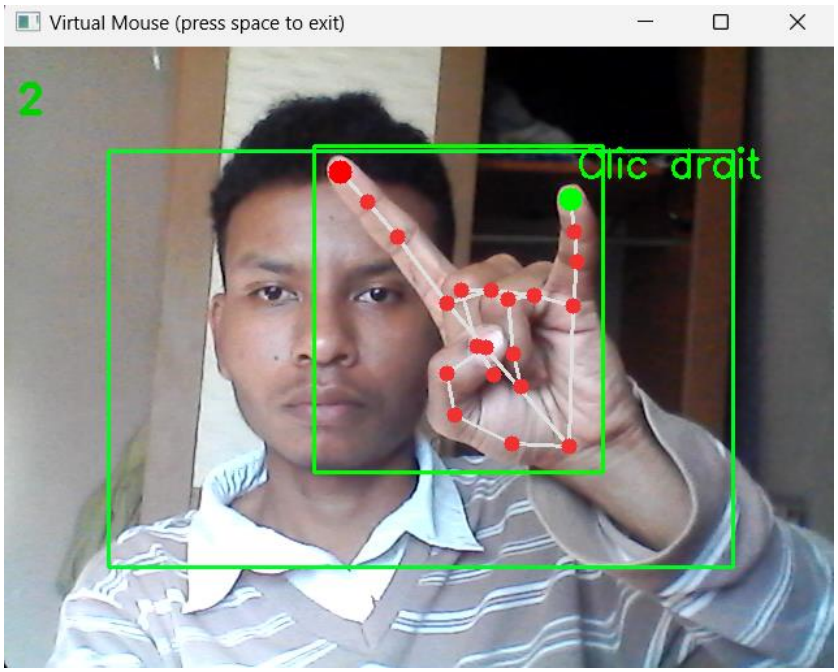
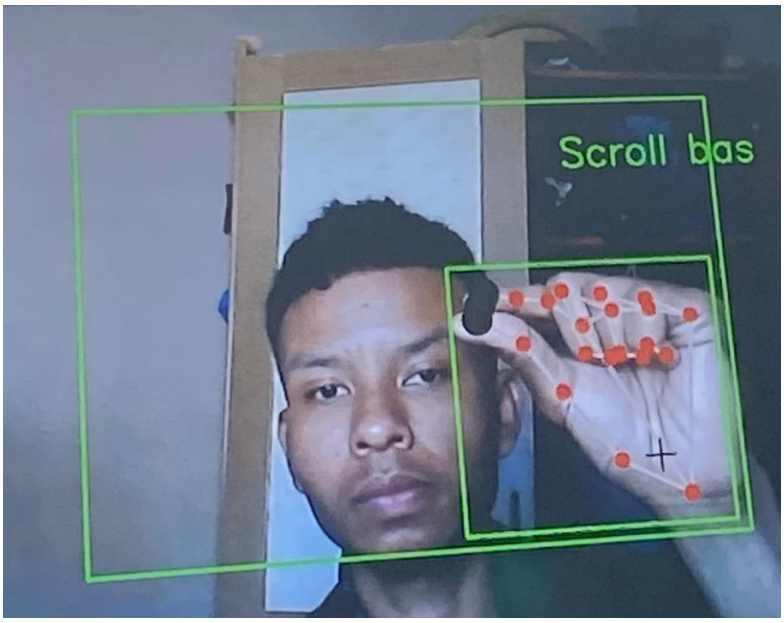
Action	Résultat
« Click droit »	
"Scroll vers le bas",	

Tableau 2.02: Résultats des tests (Suite)

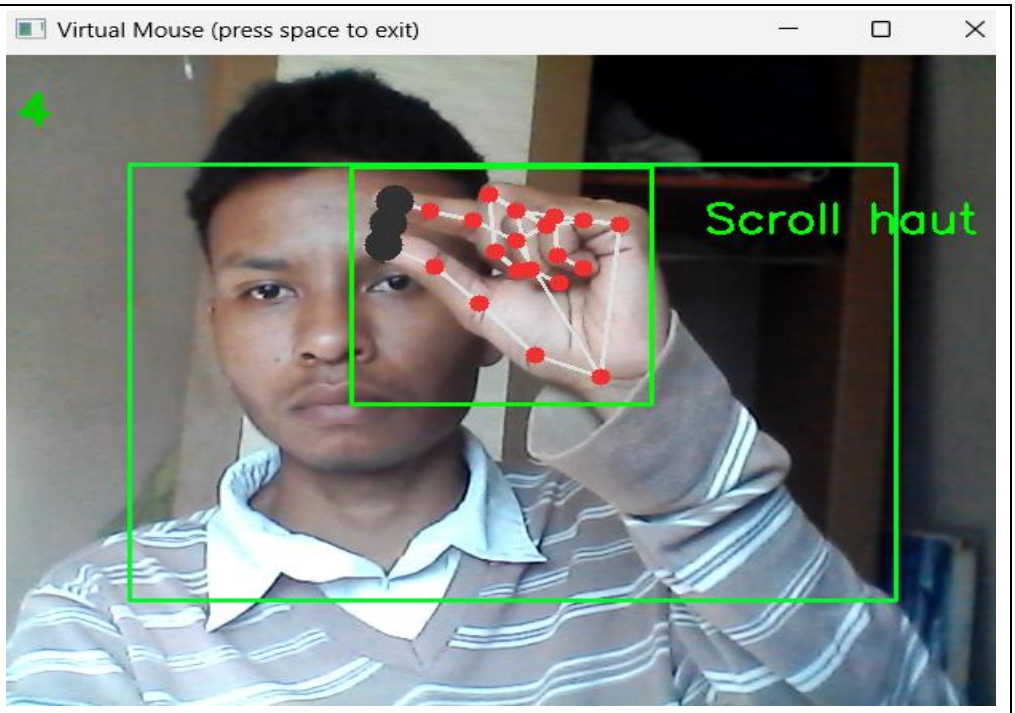
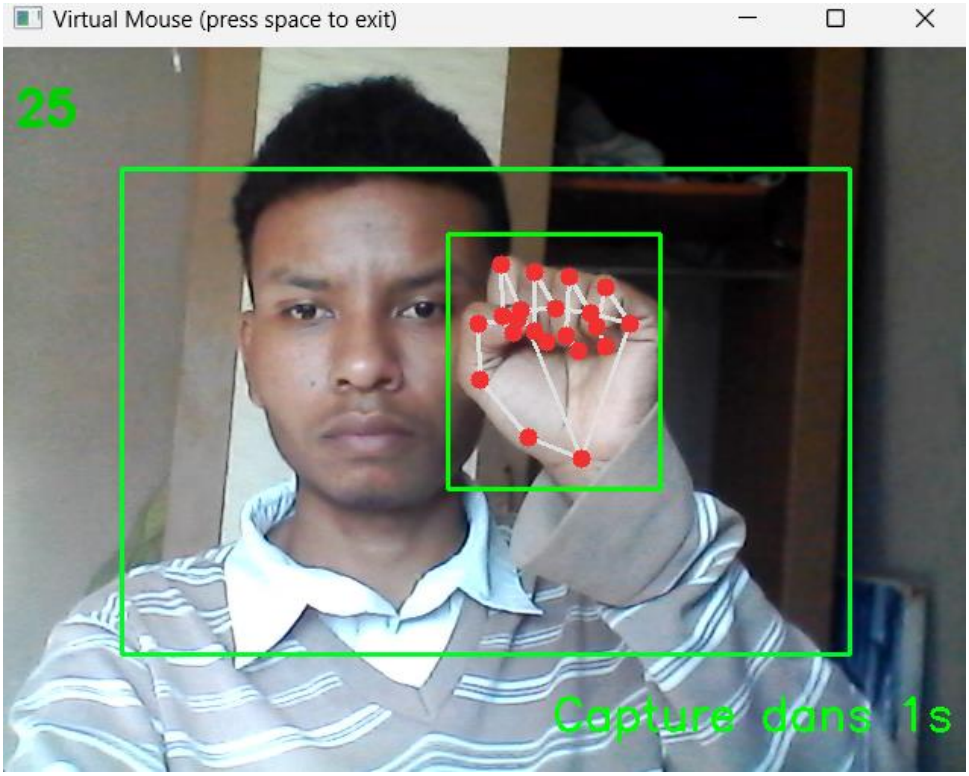
"Scroll vers le haut",	
« Capture d'écran »	

Tableau 2.02: Résultats des tests (Suite)

4.3 Métriques d'évaluation

Après le développement et l'implémentation de notre système de reconnaissance de gestes pour la souris virtuelle, l'évaluation objective des performances s'avère essentielle. Pour cela, nous

avons intégré des métriques spécifiques telles que l'analyse de la fréquence d'image en fonction du temps (FPS over time) et le temps de détection de la main en fonction du temps (Hand Detection Time over time). La première métrique, exprimée en Hz, évalue la fluidité du rendu visuel, tandis que la seconde, mesurée en millisecondes, quantifie la réactivité du système de tracking. Ces métriques fournissent des données empiriques sur les performances en temps réel de notre solution, facilitant ainsi l'identification précise des optimisations potentielles et des goulots d'étranglement dans le pipeline de traitement. Cette approche méthodologique nous permet d'évaluer rigoureusement l'efficacité de notre implémentation et d'orienter les futures itérations de développement vers une amélioration tangible de l'expérience utilisateur.

4.3.1 Analyse des Résultats

Afin de mieux comprendre les performances de notre système de contrôle par gestes, nous avons enregistré diverses interactions que des clics, des clics droits et collecté des données clés sur la fluidité et la réactivité du système comme le taux d'images par seconde (FPS) et le temps de détection des mains. Ces résultats fournissent une première base d'analyse pour évaluer l'efficacité de notre solution. Voici un récapitulatif des mesures obtenues lors d'un test récent :

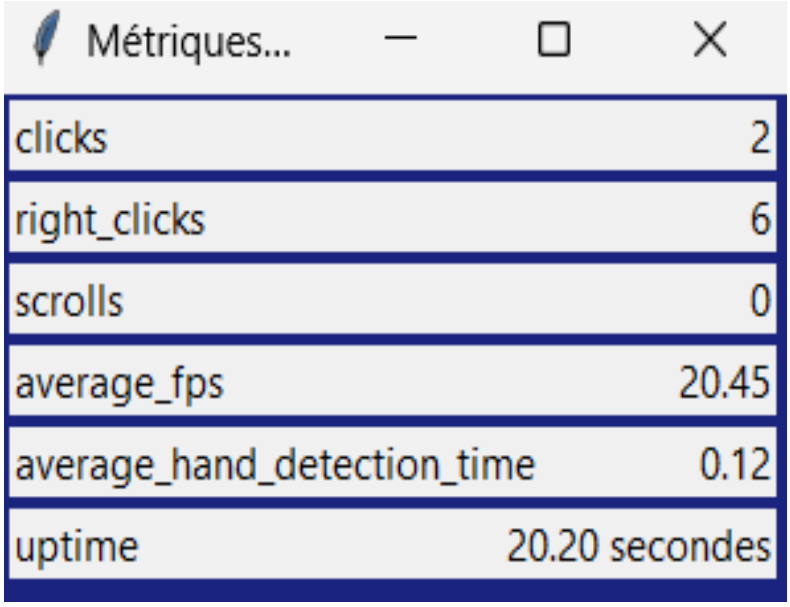


***Figure 4.06* :** Architecture de Reconnaissance des Gestes Basée sur MediaPipe

- Clicks (2) : le système a enregistré 2 clics gauches pendant la session.
- Right_clicks (6) : 6 clics droits ont été effectués.
- Scrolls (0) : Aucun défilement n'a été enregistré.

- Average_fps (20.45) : le système fonctionne en moyenne à 20.45 images par seconde, ce qui est acceptable pour une interaction en temps réel, bien que l'amélioration de ce taux pourrait rendre l'expérience plus fluide.
- Average_hand_detection_time (0.12) en moyenne, il faut 0.12 secondes (120 ms) pour détecter la main dans chaque image. C'est un temps raisonnable, mais l'optimisation de cette détection pourrait améliorer la réactivité globale.
- Uptime (20.20 secondes) : le système a fonctionné pendant environ 20 secondes lors de ce test.

Ces mesures offrent une base solide pour l'analyse des performances initiales du projet. Cependant, une évaluation plus approfondie, incluant des tests dans différents environnements et avec une variété de gestes et d'interactions, sera essentielle pour affiner et valider l'efficacité globale du système.

4.3.2 FPS over Time ou Fréquence d'Images en fonction du temps

La métrique "Images par Seconde (IPS) en fonction du temps" est essentielle pour évaluer la fluidité et la réactivité d'un système interactif en temps réel, comme c'est le cas pour une interface utilisant la détection de gestes. Le taux d'images par seconde (FPS) mesure le nombre d'images que le système est capable de traiter en une seconde. Un FPS élevé assure une expérience utilisateur fluide et réactive, tandis qu'un FPS faible peut entraîner des retards, des saccades ou une latence dans l'affichage et la détection des gestes.

Dans le contexte de notre projet de souris virtuelle basée sur la détection des mains, cette métrique est d'une importance capitale, car elle influe directement sur la qualité de l'interaction et la précision des gestes capturés.

Ici, Figure 3.04 illustre les variations temporelles de la fréquence d'images (FPS) du système de souris virtuelle, mettant en évidence plusieurs caractéristiques significatives qui reflètent ses performances dynamiques. Initialement, une ligne de base stable, oscillant autour de 6-7 FPS entre 20 et 30 secondes, indique un fonctionnement régulier dans des conditions normales. Un pic de performance soudain à 30,5 secondes, atteignant jusqu'à 35 FPS, suggère des facteurs tels qu'une optimisation dynamique du système, une diminution de la charge de traitement ou des conditions environnementales favorables. Cette hausse est suivie d'une instabilité entre 30,5 et 32 secondes, avec des fluctuations entre 15 et 35 FPS, probablement dues à des variations dans la complexité de la scène, des ajustements algorithmiques ou des interactions avec d'autres processus. La chute brutale des FPS à la fin du graphique, revenant aux niveaux de la ligne de base, pourrait indiquer un retour aux conditions normales d'utilisation ou la fin d'une

optimisation temporaire. L'amplitude des variations, allant de 6 à 35 FPS, souligne une marge d'optimisation importante, mais révèle également une sensibilité aux conditions d'utilisation, suggérant la nécessité d'une approche d'optimisation continue pour maintenir des performances stables dans diverses situations

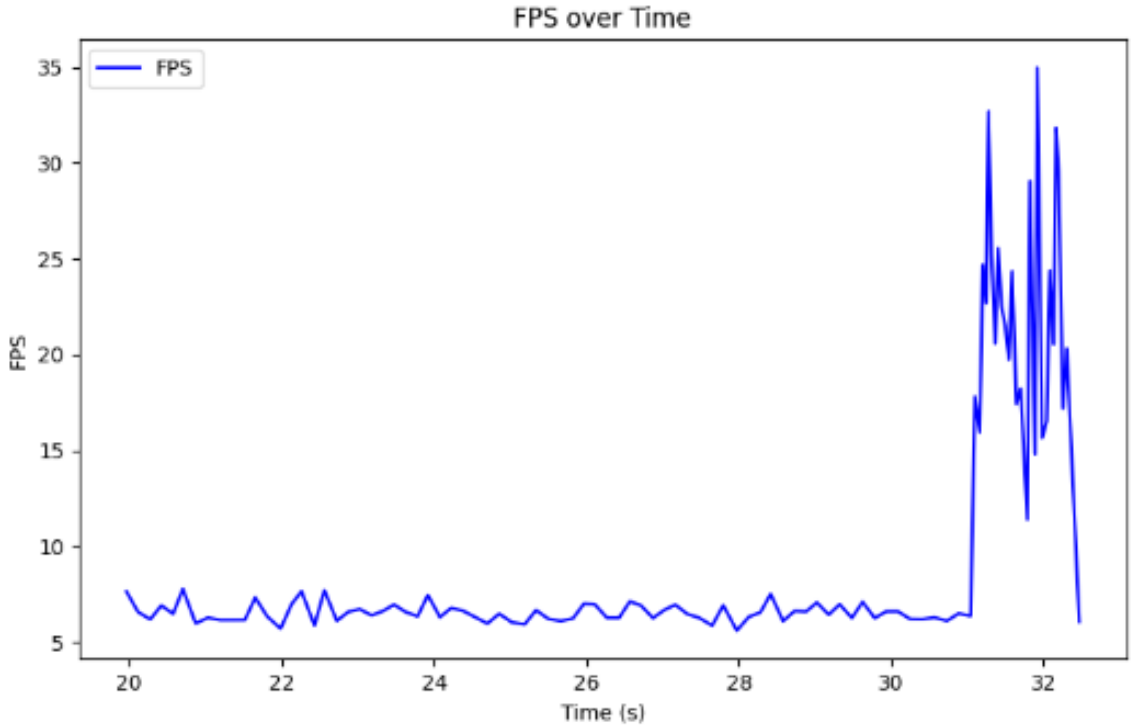


Figure 4.07 **:** Evolution temporelle de la fréquence d'images du système de souris virtuelle

4.3.3 Temps de Détection de la Main en Fonction du Temps

Le temps de détection de la main en fonction du temps est une métrique qui mesure la réactivité du système dans la reconnaissance des gestes manuels à différents moments de l'interaction. Cette métrique représente le temps nécessaire pour détecter la main dans chaque image capturée par la caméra. Un temps de détection plus faible indique une meilleure réactivité, tandis qu'une augmentation de ce temps peut signaler une surcharge du système ou des difficultés à traiter les informations visuelles en temps réel.

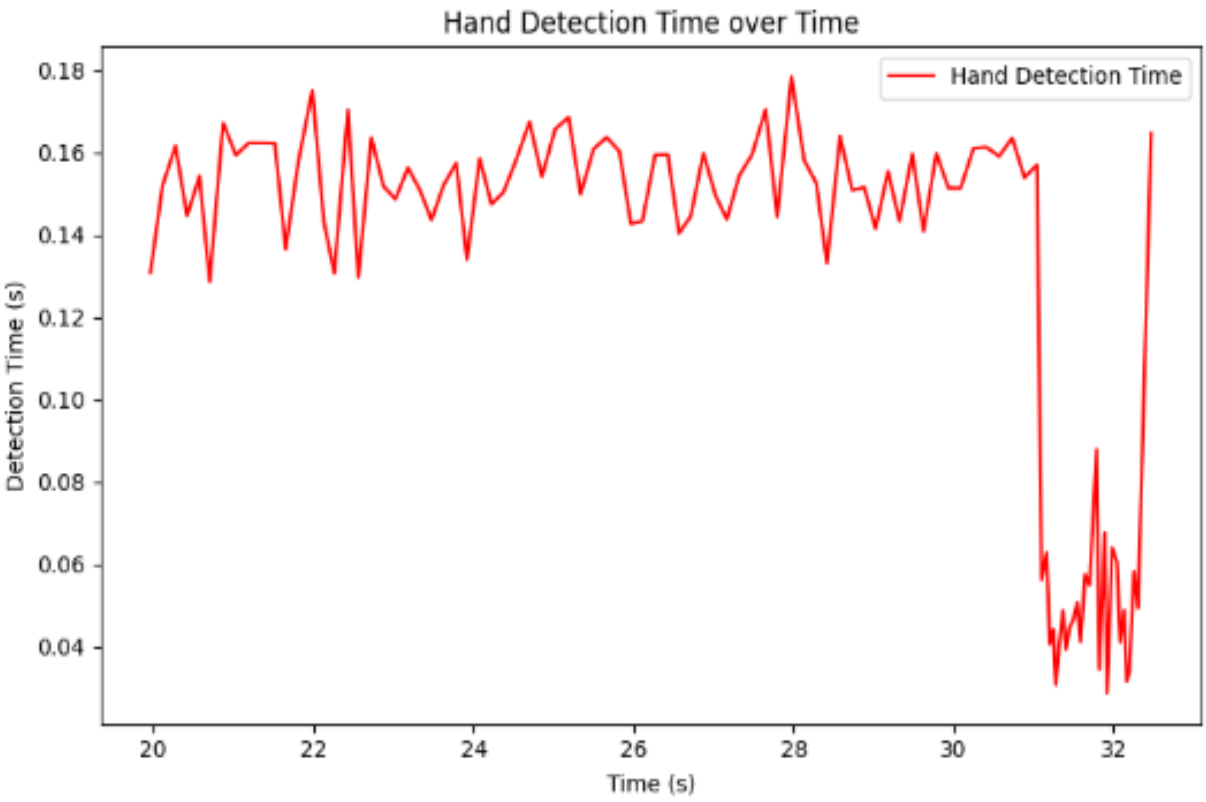


***Figure 4.08*:** Variation de la latence de détection de main dans le système de souris virtuelle

L'analyse de la *Figure 3.05* illustre l'évolution du temps de détection de la main au fil du temps dans notre système de souris virtuelle, révélant des caractéristiques cruciales de performance. On observe une période initiale (de 20 à 30 secondes) où le temps de détection oscille principalement entre 0,14 et 0,18 secondes, indiquant une stabilité relative mais avec des fluctuations notables. Cette variabilité pourrait être attribuée à des changements dans la complexité de la scène, la position de la main, ou les conditions d'éclairage. Vers 30,5 secondes, on constate une chute drastique du temps de détection, passant à environ 0,04-0,06 secondes, ce qui représente une amélioration significative de la performance. Cette amélioration soudaine pourrait être le résultat d'une optimisation dynamique de l'algorithme, d'un changement favorable dans l'environnement, ou d'une réduction temporaire de la charge de traitement. Les fluctuations rapides observées dans cette période de haute performance suggèrent que le système s'adapte en temps réel aux conditions changeantes, tout en maintenant une efficacité accrue. La brève remontée du temps de détection à la fin du graphique indique potentiellement un retour aux conditions normales ou l'introduction d'un nouveau défi de détection. Cette analyse souligne la nature dynamique et adaptative de notre algorithme de détection de main, ainsi que l'importance d'une optimisation continue pour maintenir des performances élevées et stables dans diverses conditions d'utilisation.

N.B : ll existe une corrélation entre la détection de la main (courbe rouge) et la performance du système, mesurée en FPS (courbe bleue) et en temps de détection. Lorsque le FPS diminue, cela suggère que le système consacre une plus grande part de ses ressources à la détection des gestes de la main, ce qui prolonge le temps nécessaire pour cette tâche. Inversement, lorsque le temps de détection augmente, cela coïncide souvent avec une baisse du FPS, indiquant une surcharge de calcul.

4.4 Avantages et apports du système de contrôle par gestes de la main

Le projet de reconnaissance de gestes pour le contrôle de la souris virtuelle offre plusieurs avantages significatifs qui le rendent particulièrement attractif et innovant :

- Interaction naturelle et intuitive : en permettant aux utilisateurs de contrôler leur ordinateur avec des gestes, le système offre une expérience plus naturelle et intuitive par rapport aux dispositifs traditionnels comme la souris ou le clavier. Cela simplifie l'interaction, surtout pour les utilisateurs ayant des compétences limitées en informatique.
- Accessibilité accrue : ce système apporte une solution précieuse pour les personnes souffrant de handicaps moteurs légers, en rendant possible l'interaction avec l'ordinateur par de simples mouvements de la main.
- Adaptation à divers environnements : le système peut être utilisé dans des environnements où l'utilisation de périphériques traditionnels est difficile, comme les usines, les salles d'opération, ou les laboratoires. La détection gestuelle permet de contrôler des dispositifs sans nécessiter de contact direct, garantissant ainsi une meilleure hygiène et réduisant le risque de contamination.
- Réduction des contacts physiques : dans un contexte où l’hygiène est une priorité, comme en milieu médical ou en période de pandémie, le contrôle par gestes permet de minimiser les contacts avec les surfaces, réduisant ainsi les risques de contamination.
- Optimisation continue grâce à l'IA : le projet repose sur des modèles d’apprentissage automatique et peut s’améliorer avec l’utilisation. L’algorithme peut être ajusté pour détecter des gestes plus complexes ou spécifiques, garantissant ainsi une performance accrue au fil du temps.

4.5 Limitations du Projet

Bien que le système de souris virtuelle fonctionne globalement bien, plusieurs limites demeurent, notamment en termes de performances et d'expérience utilisateur.

- Performance Inconstante : L'analyse des FPS et des temps de détection montre des fluctuations significatives. Les phases de détection intensive, marquées par une augmentation du temps de traitement, provoquent souvent une baisse des FPS. Cela entraîne des latences perceptibles, particulièrement gênantes dans les applications où la réactivité immédiate est essentielle.
- Charge de Traitement Élevée : Le système exige une utilisation importante des ressources pour la détection des mains. Cela peut poser problème dans des environnements complexes ou avec des mouvements rapides. Une charge de traitement aussi élevée réduit l'efficacité du système dans des scénarios impliquant plusieurs interactions simultanées.
- Sensibilité aux Conditions Environnementales : Les performances de détection sont affectées par l'éclairage et l'arrière-plan. Des variations dans ces conditions peuvent compromettre la précision et la fluidité des interactions. Une meilleure gestion des environnements variés améliorerait la robustesse du système.
- Limitation des Gestes : Actuellement, le système ne prend en charge qu'un ensemble restreint de gestes, limitant ainsi sa polyvalence. L'ajout de gestes supplémentaires permettrait d'étendre les cas d'usage, mais nécessiterait une optimisation accrue des algorithmes pour éviter une surcharge.

Conclusion

Le système de souris virtuelle contrôlé par des gestes de la main présente des fonctionnalités intéressantes et un potentiel prometteur dans le domaine des interfaces homme-machine. Bien qu'il fonctionne de manière satisfaisante pour des tâches de base, plusieurs défis techniques subsistent. Les fluctuations de performance, la dépendance aux conditions environnementales et la reconnaissance limitée des gestes sont autant de domaines qui nécessitent des améliorations pour garantir une expérience utilisateur fluide et optimisée.

CONCLUSION GENERALES

Pour conclure, ce mémoire a permis d'explorer les technologies de vision artificielle et d'apprentissage profond appliquées à la reconnaissance de gestes de la main, dans le but de concevoir un système interactif homme-machine contrôlé par des gestes. La conception s'est déroulée en plusieurs phases clés, débutant par l'analyse des besoins fonctionnels et techniques du système, suivie de la modélisation de l'architecture, de la sélection des outils logiciels, puis de la phase de développement et d'implémentation.

Le système IHM (Interface Homme-Machine) repose sur une interface fluide où les gestes sont capturés en temps réel via une webcam, traités par des algorithmes de vision artificielle, et convertis en actions interactives. Nous avons utilisé des outils modernes tels qu'OpenCV et MediaPipe pour la détection des gestes, ainsi que des réseaux de neurones convolutifs pour la reconnaissance. Les résultats ont montré une reconnaissance précise des gestes avec un bon taux de détection en temps réel, ce qui justifie l'affirmation que "le système développé a démontré une efficacité notable". En effet, les tests ont révélé des performances robustes même dans des conditions variées, telles que des changements de luminosité modérés ou des gestes complexes.

Il serait pertinent d'ajouter à ce travail des techniques d'apprentissage plus avancées, telles que les réseaux récurrents, pour mieux gérer les mouvements rapides ou les gestes plus complexes. De plus, l'intégration de caméras stéréo ou de capteurs de profondeur permettrait de détecter avec plus de précision les dimensions et la distance des gestes.

Les étapes les plus difficiles lors de la réalisation ont été principalement liées à l'optimisation des algorithmes pour assurer un traitement en temps réel tout en maintenant une précision élevée. La gestion des variations de lumière et des mouvements rapides a également présenté des défis, nécessitant plusieurs itérations d'ajustements techniques.

La phase de conception était axée sur la théorie et les choix techniques : définir l'architecture du système et sélectionner les technologies adaptées. En revanche, la phase de réalisation a demandé un travail pratique plus intensif, notamment dans l'intégration des algorithmes et la gestion des imprévus liés à la capture en temps réel des gestes.

Suite à l'évaluation des performances du système, nous avons retenu que le modèle de reconnaissance atteint un niveau d'efficacité satisfaisant pour des applications pratiques, bien que des améliorations futures pourraient encore optimiser sa robustesse. En résumé, ce projet a prouvé la faisabilité et l'efficacité de la reconnaissance gestuelle pour des interfaces homme-machine, tout en ouvrant la voie à des développements futurs prometteurs.

ANNEXES

ANNEXES 1

NOTIONS ESSENTIELLES

Anatomie et physiologie de l'œil humain

L'œil humain est l'organe de la vision de l'être humain ; il lui permet de capter la lumière, pour ensuite l'analyser et interagir avec son environnement. L'œil humain permet de distinguer les formes et les couleurs. L'un des grands défis de la technologie sera de fabriquer des yeux électroniques, capables d'égaler voire de dépasser les aptitudes des yeux du monde vivant pour, par exemple, remplacer l'œil d'une personne accidentée. L'œil humain est constitué d'un globe oculaire comportant :

- Sur sa partie antérieure, la cornée, qui est une calotte sphérique transparente :
- Sur le reste du globe, la sclère, ou sclérotique, qui forme le « blanc » de l'œil.

Le globe oculaire mesure environ 2,5 cm de diamètre et a une masse de 8 grammes. Il est formé de 3 enveloppes, ou tuniques, entourant une substance gélatineuse appelée le corps vitré. Les trois tuniques s'appellent la tunique externe, la tunique moyenne, et la tunique interne ; le corps vitré est principalement constitué d'eau et sert à maintenir la forme de l'œil.

Anatomie de l'œil humain

L'œil s'adapte en premier lieu à la lumière ambiante. L'être humain peut ainsi percevoir avec une sensibilité équivalente en plein soleil ou sous la lumière de la pleine lune, soit avec une intensité lumineuse 10 000 fois moindre. Une première adaptation provient de l'écartement de l'iris qui, en mode nocturne, peut atteindre une ouverture maximale de 7 mm pour des jeunes gens (maximum qui décroît à 4 mm avec l'âge).

ANNEXE 2

CODE SOURCE DE LA RECONNAISSANCE DE LA MAIN

A1.1 Détecter les points clés de la main

```
import cv2
import mediapipe as mp
class HandDetector():
    def __init__(self, mode=False, maxHands=2, detectConf=0.5, trackConf=0.5):
```

```
        self.mode = mode
        self.maxHands = maxHands
        self.detectConf = detectConf
        self.trackConf = trackConf
        # Initialiser MediaPipe Hands
        self.mpHands = mp.solutions.hands
        self.hands = self.mpHands.Hands(self.mode, max_num_hands=self.maxHands,
                                        min_detection_confidence=self.detectConf,
                                        min_tracking_confidence=self.trackConf)
        self.mpDraw = mp.solutions.drawing_utils
    # Détecter les mains et leurs landmarks
    def findHands(self, img, draw=True):
        imgRGB = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        self.results = self.hands.process(imgRGB)
        if self.results.multi_hand_landmarks:
            for handLms in self.results.multi_hand_landmarks:
                if draw:
                    self.mpDraw.draw_landmarks(img,
handLms, self.mpHands.HAND_CONNECTIONS)
        return img
    # Trouver les positions des landmarks (points clés)
    def findPosition(self, img, handNum=0, draw=True):
        self.lmList = []
        if self.results.multi_hand_landmarks:
            myHand = self.results.multi_hand_landmarks[handNum]
            h, w, c = img.shape
            for id, lm in enumerate(myHand.landmark):
                cx, cy = int(lm.x * w), int(lm.y * h)
                self.lmList.append([id, cx, cy])
                if draw:
```

```
                cv2.circle(img, (cx, cy), 6, (50, 50, 240), cv2.FILLED)
        return self.lmList
```

A1.2 Reconnaitre les gestes de la main

```
 class HandDetector():
    def __init__(self):
        self.tipIds = [4, 8, 12, 16, 20]
    # Reconnaître quel doigt est levé
    def fingersUp(self):
        fingers = []
        tipIds = self.tipIds
        lmList = self.lmLis
        # Pouce
        if lmList[tipIds[0]][1] < lmList[tipIds[0] - 1][1]:  # Comparer les positions du pouce
            fingers.append(1)  # Pouce levé
        else:
            fingers.append(0)  # Pouce baissé
        # Autres doigts
        for id in range(1, 5):
            if lmList[tipIds[id]][2] < lmList[tipIds[id] - 2][2]:  # Comparer les positions des autres
doigts
       fingers.append(1)  # Doigt levé
            else:
          fingers.append(0)  # Doigt baissé
        return fingers
```

ANNEXE 2

CODE SOURCE SUR LE SOURIS VIRTUEL

A2.1 Importer les packages nécessaires

```
import numpy as np
import pyautogui
```

```
from functions.HandTrackingModuleVM import HandDetector
```

A2.2 Suivi du mouvement de la souris avec le doigt

```
def virtual_mouse(num_cam, width_cam, height_cam):
    frameR = 80  # Zone réduite pour éviter les bords
    smoothening = 3  # Lissage des mouvements
    pyautogui.FAILSAFE = False
    # Configuration de la caméra et de la résolution
    cap = cv2.VideoCapture(num_cam)
    cap.set(cv2.CAP_PROP_FRAME_WIDTH, width_cam)
    cap.set(cv2.CAP_PROP_FRAME_HEIGHT, height_cam)

    # Taille de l'écran
    wScreen, hScreen = pyautogui.size()
    detector = HandDetector(maxHands=1)  # Détection d'une seule main
    plocX, plocY = 0, 0  # Position précédente de la souris
    currX, currY = 0, 0  # Position actuelle de la souris
    while True:
        success, img = cap.read()
        img = cv2.flip(img, 1)  # Inverser l'image pour correspondre aux
mouvements
        img = detector.findHands(img)
        lmList, _ = detector.findPosition(img)
        if lmList:
            x1, y1 = lmList[8][1:]  # Coordonnées de l'index
            # Vérifier si seul l'index est levé
            fingers = detector.fingersUp()
            if fingers[1] == 1 and fingers[2] == 0:
                # Convertir les coordonnées pour correspondre à l'écran
                x3 = np.interp(x1, (frameR, width_cam - frameR), (0, wScreen))
```

```
y3 = np.interp(y1, (frameR, height_cam - frameR), (0, hScreen))
# Lissage des mouvements
currX = plocX + (x3 - plocX) / smoothening
currY = plocY + (y3 - plocY) / smoothening
# Déplacer la souris
pyautogui.moveTo(currX, currY)
plocX, plocY = currX, currY
```

A2.3 Clic gauche , clic droit et Défilement (scroll)

```
# Clic gauche (index et majeur rapprochés)
if fingers[1] == 1 and fingers[2] == 1:
length, img, lineInfo = detector.findDistance(8, 12, img)
if length < 40:  # Distance entre l'index et le majeur
pyautogui.click()
# Clic droit (index et auriculaire levés)
if fingers[1] == 1 and fingers[4] == 1 and fingers[2] == 0 and fingers[3] == 0:
pyautogui.rightClick()
# Scroll (pouce et index rapprochés)
if fingers[0] == 1 and fingers[1] == 1:
length, img, lineInfo = detector.findDistance(4, 8, img)
if length < 20:
pyautogui.scroll(-50)  # Scroll vers le bas
elif 20 < length < 40:
pyautogui.scroll(50)  # Scroll vers le haut
```

ANNEXE 3

CODE SOURCE SUR LA CREATION DE L'INTERFACE

A3.1 Importer les packages nécessaires

```
import tkinter as tk
from tkinter import ttk, simpledialog, messagebox
from PIL import Image, ImageTk
```

```
import threading
import cv2
```

A3.2 Création de l'interface principale

```
#Création de la fenêtre principale
root = tk.Tk()
root.title("Souris Virtuelle")
# Agrandir l'interface (ex: 400x400)
centrer_fenetre(root, largeur=400, hauteur=400)
# Appliquer le thème par défaut
theme_application(root)
# Charger l'image et la redimensionner
image = Image.open("./ressources/logo.png")
image = image.resize((100, 100), Image.LANCZOS)
image = ImageTk.PhotoImage(image)
# Créer un label avec l'image
etiquette_image = tk.Label(root, image=image, bg='#1a237e')
etiquette_image.pack(pady=20)
# Créer un style pour les boutons
style = ttk.Style()
style.configure('TButton', font=('Inter', 12), background='#3f51b5',
foreground='black')
```

BIBLIOGRAPHIE

[1] C. Cadoz « *Le geste canal de communication homme/machine – la communication instrumentale. Techniques et Science Informatiques* », 1994

[2] B. Chahrazed, « *Système de Reconnaissance des gestes de la main* », Université Abou Bekr Belkaid 2018.

[3] A. Cheloufi, « *Système de Reconnaissance des gestes de la main* », Université Abou Bekr Belkaid 2014.

[4] J. Bernd & H. Horst, (2000). « *Computer Vision and Applications, A Guide for Students and Practitioners* ». Academic Press.

[5] Konfuzio. (s.d.). Qu'est-ce que la vision par ordinateur ? La perception au moyen de l'IA expliquée simplement. https://konfuzio.com/fr/vision-par-ordinateur.

[6] Shapiro, G., & Stockman, C. (2001). « *Computer Vision. Prentice Hall* ».

[7] IBM. (s.d.). Qu'est-ce que la Computer Vision ? Qu'est-ce que la vision par ordinateur ? | IBM consulté le 16 Septembre 2024.

[8] M. H. Ramafiarisona, (2023-2024). Traitement d'image. Note CoursLP3 TCO, Ment. TCO - E.S.P.A.

[9] Y. Meng , H. Jiang , N. Duan and H. Wen. « Real-Time Hand Gesture Monitoring Model Based on MediaPipe&rsquos Registerable System » (mdpi.com) , (2024).

[10] J. Hertz, R. Palmer, & A. Krogh, (1990). « *Introduction to the theory of neural computation. Perseus Books* ».

[11] Groupe Institut National des sciences appliquées (INSA), « *Initiez-vous au Deep Learning* », Cours en ligne, Coursera, https://openclassrooms.com/fr/courses/5801891-initiez-vous-au-deep-learning, 2024.

[12] T.Voillemin ,« *Réseaux de Neurones à Convolution Spatio-Temporelle pour l'analyse et la reconnaissance précoce d'actions et de gestes* », Ecole Doctorale SPI-MADIS, Université Lille, 2021, **https://theses.hal.science/tel-03557617v2**

[13] ARNAUD BODIN & FRANÇOIS RECHER, Deep math « *Mathématiques (simples) des réseaux de neurones (pas trop compliqués)* », 2023

[14] S. Russell, P. Norvig, « *Artificial Intelligence, A Modern Approach* », Pearson Education: New Jersey, 2010.

[15] K. Gurney, (1997). « *An introduction to neural networks* ».

[16] H. Hiyadi, (2016). « *Reconnaissance 3D de gestes pour l'interaction homme-système* ». Thèse présentée et soutenue à Rabat. [23] Dupont, J. (2021). Reconnaissance optique et intelligente des caractères : applications et enjeux. Université de Paris.

FICHE DE RENSEIGNEMENTS

Nom : RAMANANANDRO

Prénom(s) : Alex Mathieu

Adresse de l'auteur : Bloc 11 Porte 386 Cur Vontovorona
Manakambahiny V015C

Téléphone : +261 34 31 098 42

E-mail : ramananandrom@gmail.com

Titre du mémoire :

« CONCEPTION D'UNE INTERFACE HOMME-MACHINE CONTRÔLÉE PAR DES GESTES DE LA MAIN »

Nombre de pages : 68

Nombre de tableaux : 2

Nombre de figures : 42

Directeur de mémoire

Nom : BOTO ANDRIANANDRASANA Jean Espérant

E-mail : jean.respira@gmail.com

Téléphone : +261 34 04 281 43

RESUME

Ce mémoire porte sur la conception et la mise en œuvre d'une interface homme-machine contrôlée par des gestes de la main, exploitant les dernières avancées en vision artificielle et en apprentissage profond. À travers l'utilisation d'outils tels que OpenCV et MediaPipe, le système est capable de capturer et interpréter des gestes en temps réel, ouvrant la voie à des applications variées comme le contrôle de robots, l'interaction avec des jeux vidéo, ou encore l'assistance aux personnes en situation de handicap. L'étude présente les concepts de base de la reconnaissance gestuelle, les architectures de réseaux neuronaux convolutifs employées pour la classification, et l'intégration des techniques de traitement d'image. Les résultats expérimentaux sont prometteurs, montrant une reconnaissance précise des gestes dans des conditions variées. Ce travail constitue une base solide pour des recherches futures sur les interfaces gestuelles, tout en suggérant des améliorations potentielles en matière d'efficacité et de robustesse du système.

Mots clés : Gestuelle, Vision artificielle, Apprentissage profond, Interface homme-machine, convolution.

ABSTRACT

This research thesis focuses on the design and implementation of a human-machine interface controlled by hand gestures, leveraging the latest advances in computer vision and deep learning. Using tools such as OpenCV and MediaPipe, the system captures and interprets gestures in real-time, paving the way for diverse applications such as robot control, video game interaction, and assistance for people with disabilities. The study covers the fundamental concepts of gesture recognition, the use of convolutional neural networks for classification, and the integration of image processing techniques. The experimental results are promising, demonstrating accurate gesture recognition in various conditions. This work provides a solid foundation for future research in gesture-based interfaces, with potential improvements suggested to enhance the system's efficiency and robustness.

Keywords: Gesture, Computer vision, Deep learning, Human-machine interface, Convolution.